```
// ========= File: src/game_layout.h ==========
// src/game_layout.h
#ifndef GAME_LAYOUT_H
#define GAME_LAYOUT_H

#include "splashkit.h"
const int TEXT_CHAR_WIDTH = 8;

// All screen positions and sizes.
const int WINDOW_WIDTH = 960;
const int WINDOW_HEIGHT = 700;

// The 4 x 4 board.
const int BOARD_X = 54;
const int BOARD_Y = 160;
const int CELL_SIZE = 112;
const int CELL_GAP = 10;
const int BOARD_SIZE = 4 * CELL_SIZE + 3 * CELL_GAP;

// Food pictures.
const int FOOD_IMAGE_SIZE = 96;
const int FOOD_MARGIN = (CELL_SIZE - FOOD_IMAGE_SIZE) / 2;

// The three generator buttons sit directly under the board.
const int GENERATOR_Y = BOARD_Y + BOARD_SIZE + 4;
const int GENERATOR_WIDTH = 150;
const int GENERATOR_HEIGHT = 54;
const int GENERATOR_GAP = 14;
const int GENERATOR_ICON_SIZE = 46;
const int DOUGH_BUTTON_X = BOARD_X;
const int TART_BUTTON_X = BOARD_X + GENERATOR_WIDTH + GENERATOR_GAP;
const int CAKE_BUTTON_X = BOARD_X + 2 * (GENERATOR_WIDTH + GENERATOR_GAP);

// Back button in the top-right corner of the play screen.
const int BACK_X = 825;
const int BACK_Y = 14;
const int BACK_WIDTH = 110;
const int BACK_HEIGHT = 36;

// Pause control in the play-screen header.
const int PAUSE_BUTTON_X = 825;
const int PAUSE_BUTTON_Y = 60;
const int PAUSE_BUTTON_WIDTH = 110;
const int PAUSE_BUTTON_HEIGHT = 36;

// Right-hand column of the play screen: customer picture, order, hearts, tips.
const int RIGHT_X = 552;
const int RIGHT_WIDTH = 378;
const int CUSTOMER_BOX_Y = 150;
const int CUSTOMER_BOX_WIDTH = 146;
```

```
const int CUSTOMER_BOX_HEIGHT = 144;
const int ORDER_BOX_X = RIGHT_X + CUSTOMER_BOX_WIDTH + 8;
const int ORDER_BOX_WIDTH = RIGHT_X + RIGHT_WIDTH - ORDER_BOX_X;

// The status panel under the order.
const int STATUS_BOX_Y = 310;
const int STATUS_BOX_HEIGHT = 135;

// Settings button (top-right of the mode-selection screen) and the settings panel.
const int SETTINGS_BUTTON_X = 825;
const int SETTINGS_BUTTON_Y = 14;
const int SETTINGS_BUTTON_WIDTH = 110;
const int SETTINGS_BUTTON_HEIGHT = 36;
const int SETTINGS_BOX_X = 280;
const int SETTINGS_BOX_Y = 170;
const int SETTINGS_BOX_WIDTH = 400;
const int SETTINGS_BOX_HEIGHT = 310;
const int SETTINGS_TOGGLE_X = SETTINGS_BOX_X + 270;      // the ON/OFF buttons
const int SETTINGS_TOGGLE_WIDTH = 100;
const int SETTINGS_TOGGLE_HEIGHT = 40;
const int SETTINGS_MUSIC_Y = SETTINGS_BOX_Y + 80;
const int SETTINGS_EFFECTS_Y = SETTINGS_BOX_Y + 140;
const int SETTINGS_CLOSE_X = SETTINGS_BOX_X + 120;
const int SETTINGS_CLOSE_Y = SETTINGS_BOX_Y + 240;
const int SETTINGS_CLOSE_WIDTH = 160;
const int SETTINGS_CLOSE_HEIGHT = 44;

// Name plate under the story character.
const int STORY_NAME_X = 52;
const int STORY_NAME_Y = 424;
const int STORY_NAME_WIDTH = 223;
const int STORY_NAME_HEIGHT = 34;

// The four mode cards on the mode-selection screen.
const int MODE_CARD_WIDTH = 310;
const int MODE_CARD_HEIGHT = 137;
const int MODE_LEFT_X = 150;
const int MODE_RIGHT_X = 500;
const int MODE_TOP_Y = 165;
const int MODE_BOTTOM_Y = 335;

// The panel behind the text on the win and game-over screens.
const int END_BOX_X = 160;
const int END_BOX_Y = 200;
const int END_BOX_WIDTH = 640;
const int END_BOX_HEIGHT = 250;

// Buttons shown after a round ends.
const int END_REPLAY_BUTTON_X = 300;
const int END_MENU_BUTTON_X = 490;
```

```
const int END_BUTTON_Y = END_BOX_Y + 190;
const int END_BUTTON_WIDTH = 170;
const int END_BUTTON_HEIGHT = 44;
const int PAUSE_PANEL_X = 280;
const int PAUSE_PANEL_Y = 265;
const int PAUSE_PANEL_WIDTH = 400;
const int PAUSE_PANEL_HEIGHT = 140;


#endif

// ========= File: src/utilities.h ==========
// src/utilities.h
#ifndef UTILITIES_H
#define UTILITIES_H

#include "splashkit.h"

// Small helper functions shared by the game and the checks.

// Turn a whole number into text for drawing on screen, e.g. 42 -> "42".
string number_text(int number);

// True when min <= value <= max (the idea behind read_integer_range without any typing).
bool is_in_range(int value, int min, int max);

// True when the point (x, y) is inside the rectangle that starts at (rect_x, rect_y) and is width pixels wide and height pixels high.
bool point_in_rectangle(double x, double y, int rect_x, int rect_y, int width, int height);

// Estimated width in pixels of a line of text.
int estimated_text_width(string text);

// The x position that centres a line of text inside a box.
int centered_text_x(string text, int box_x, int box_width);

#endif

// ========= File: src/utilities.cpp ==========
// src/utilities.cpp
#include "splashkit.h"
#include "utilities.h"
#include "game_layout.h"

string number_text(int number)
{
    return to_string(number);
}

bool is_in_range(int value, int min, int max)
```

```
{
    return value >= min && value <= max;
}

bool point_in_rectangle(double x, double y, int rect_x, int rect_y, int width, int height)
{
    return x >= rect_x && x < rect_x + width && y >= rect_y && y < rect_y + height;
}

int estimated_text_width(string text)
{
    return (int)text.length() * TEXT_CHAR_WIDTH;
}

int centered_text_x(string text, int box_x, int box_width)
{
    return box_x + (box_width - estimated_text_width(text)) / 2;
}

// ========= File: src/game_state.h ==========
// src/game_state.h
#ifndef GAME_STATE_H
#define GAME_STATE_H

#include "splashkit.h"

const int GRID_SIZE = 4;
// The food atlas has four images per category, using tiers 0 through 3.
const int MAX_ITEM_TIER = 3;
const int ORDER_COUNT = 3;
const int CLASSIC_ORDER_COUNT = 5;
const int MAX_ORDER_COUNT = CLASSIC_ORDER_COUNT;

// Each food chain has its own category.
enum item_category
{
    NO_ITEM,
    DOUGH,
    TART,
    CAKE
};

// Screens and game modes.
enum game_mode
{
    BEGINNER_MODE,
    CLASSIC_MODE,
    RUSH_MODE,
    RECIPE_MODE
};
```

```
200
201 int order_count_for_mode(game_mode mode);
202
203 enum screen_state
204 {
205     MODE_MENU,
206     STORY_SCREEN,
207     PLAYING_SCREEN,
208     WON_SCREEN,
209     GAME_OVER_SCREEN
210 };
211
212 // A board square stores both the food category and its tier.
213 struct board_item
214 {
215     item_category category;
216     int tier;
217 };
218
219 struct order_spec
220 {
221     item_category category;
222     int tier;
223     int quantity;
224     item_category secondary_category;
225     int secondary_tier;
226     int secondary_quantity;
227     int move_limit;
228 };
229
230 struct game_state
231 {
232     // A square with category NO_ITEM is empty.
233     board_item board[GRID_SIZE][GRID_SIZE];
234     int selected_row, selected_column;
235     int score, moves, order_index, delivered_count, satisfaction;
236     int recipe_first_delivered, recipe_second_delivered, moves_left;
237     order_spec orders[MAX_ORDER_COUNT];
238     unsigned int order_started_at;
239     string last_action;
240
241     game_mode mode;
242     screen_state screen;
243
244     int story_line;
245     bool lost_by_timer, lost_by_move_limit, low_time_alert_played;
246
247     // True while "Order complete" is showing and the customer is leaving.
248     // The board is locked and the Rush Hour timer is paused until it ends.
249     bool order_just_completed;
```

```
    bool paused;
    unsigned int paused_at;
    unsigned int order_completed_at;
    bool served_food_removed;   // true once the served food has been taken off the board
    bool recipe_food_pending[GRID_SIZE][GRID_SIZE];
    unsigned int recipe_food_delivered_at[GRID_SIZE][GRID_SIZE];

    // Player settings. Starting a new round does NOT reset these, so they
    // stay the same until the player changes them.
    bool music_on;
    bool sound_effects_on;
    bool settings_open;        // true while the settings panel is showing
    bool new_high_score;        // true when this round sets a new local record
    bool result_recorded;      // prevents saving the same round more than once
};

// Program start: reset everything and show the story first.
void start_new_game(game_state &game);
// Reset everything and show the four-mode selection screen.
void show_mode_menu(game_state &game);
// Start the chosen mode's gameplay immediately.
void choose_mode(game_state &game, game_mode mode, unsigned int now);
item_category current_order_category(const game_state &game);
item_category current_order_secondary_category(const game_state &game);
bool generate_item(game_state &game, item_category category);

// Settings panel on the mode-selection screen. Music and sound effects
// are two separate switches.
void open_settings(game_state &game);
void close_settings(game_state &game);
void toggle_music(game_state &game);
void toggle_sound_effects(game_state &game);

#endif

// ========= File: src/game_state.cpp ==========
// src/game_state.cpp
#include "splashkit.h"
#include "game_state.h"
#include "sound_manager.h"

int order_count_for_mode(game_mode mode)
{
    return (mode == CLASSIC_MODE) ? CLASSIC_ORDER_COUNT : ORDER_COUNT;
}

// Generate one shuffled pass through all three food categories.
static void shuffled_categories(item_category categories[3])
{
    categories[0] = DOUGH;
```

```
    categories[1] = TART;
    categories[2] = CAKE;
    for (int i = 2; i > 0; i--)
    {
        int swap_index = rnd(i + 1);   // rnd(n) gives 0 to n - 1, so this is 0 to i
        item_category temp = categories[i];
        categories[i] = categories[swap_index];
        categories[swap_index] = temp;
    }
}

// Store the orders once when a mode starts so they stay stable during play.
static void setup_orders(game_state &game, game_mode mode)
{
    item_category categories[3];
    if (mode == BEGINNER_MODE)
    {
        categories[0] = DOUGH;
        categories[1] = TART;
        categories[2] = CAKE;
    }
    else
    {
        shuffled_categories(categories);
    }

    for (int i = 0; i < order_count_for_mode(mode); i++)
    {
        order_spec &order = game.orders[i];
        order.category = (mode == BEGINNER_MODE) ? (item_category)(DOUGH + i) : categories[i
% ORDER_COUNT];
        order.tier = (mode == BEGINNER_MODE) ? ((i == 2) ? 3 : 2) : 1 + rnd(2);
        order.quantity = 1;
        order.secondary_category = NO_ITEM;
        order.secondary_tier = -1;
        order.secondary_quantity = 0;
        order.move_limit = 12 + i * 4;

        if (mode == CLASSIC_MODE)
            order.quantity = 1 + rnd(2);   // 1 or 2 items
        else if (mode == RECIPE_MODE)
        {
            order.quantity = 1 + rnd(2);   // 1 or 2 items of the first food
            order.secondary_category = categories[(i + 1) % ORDER_COUNT];
            order.secondary_tier = 1 + rnd(2);
            order.secondary_quantity = 1;
        }
    }
}

```

```
// Empty every square of the board.
static void setup_board(game_state &game)
{
    for (int row = 0; row < GRID_SIZE; row++)
    {
        for (int column = 0; column < GRID_SIZE; column++)
        {
            game.board[row][column].category = NO_ITEM;
            game.board[row][column].tier = 0;
            game.recipe_food_pending[row][column] = false;
            game.recipe_food_delivered_at[row][column] = 0;
        }
    }
}

// Reset every value that changes during a round.
static void reset_round(game_state &game)
{
    game.mode = BEGINNER_MODE;
    game.score = 0;
    game.moves = 0;
    game.order_index = 0;
    game.delivered_count = 0;
    game.satisfaction = 3;
    game.recipe_first_delivered = 0;
    game.recipe_second_delivered = 0;
    game.moves_left = 0;
    game.story_line = 0;
    game.order_started_at = 0;
    game.lost_by_timer = false;
    game.lost_by_move_limit = false;
    game.low_time_alert_played = false;
    game.order_just_completed = false;
    game.paused = false;
    game.paused_at = 0;
    game.order_completed_at = 0;
    game.served_food_removed = false;
    game.last_action = "";
    game.settings_open = false;   // the panel is closed, but music/effects settings are
kept
    game.new_high_score = false;
    game.result_recorded = false;
    game.selected_row = -1;
    game.selected_column = -1;
    setup_orders(game, BEGINNER_MODE);
    setup_board(game);
}

// Called once when the program starts: the story scene comes first.
void start_new_game(game_state &game)
```

```
{
    reset_round(game);
    game.music_on = true;           // both switches start ON; reset_round never changes them
    game.sound_effects_on = true;
    game.screen = STORY_SCREEN;
}

// Used by the Back button, the R key and the end screens.
// The story is not shown again.
void show_mode_menu(game_state &game)
{
    if (game.paused)
        apply_music_setting(game);
    reset_round(game);
    game.screen = MODE_MENU;
}

// Start the selected mode and go straight to gameplay.
void choose_mode(game_state &game, game_mode mode, unsigned int now)
{
    reset_round(game);
    game.mode = mode;
    setup_orders(game, mode);
    game.moves_left = (mode == RECIPE_MODE) ? game.orders[0].move_limit : 0;
    game.order_started_at = now;
    game.screen = PLAYING_SCREEN;
}

void open_settings(game_state &game)
{
    game.settings_open = true;
}

void close_settings(game_state &game)
{
    game.settings_open = false;
}

// Each switch only changes its own setting.
void toggle_music(game_state &game)
{
    game.music_on = !game.music_on;
}

void toggle_sound_effects(game_state &game)
{
    game.sound_effects_on = !game.sound_effects_on;
}

```

```
// ========= File: src/game_logic.h ==========
// src/game_logic.h
#ifndef GAME_LOGIC_H
#define GAME_LOGIC_H

#include "splashkit.h"
#include "game_state.h"

// Rush Hour gives the player 30 seconds to complete each order.
const unsigned int ORDER_TIME_LIMIT = 30000;

// After an order is finished (times in milliseconds):
// 0 - 1000: "Order complete" message, customer still shown
// 1000 - 1600: customer has left, waiting for the next one
// 1600: next customer arrives with a new order
const unsigned int ORDER_COMPLETE_MESSAGE_TIME = 1000;

// Rush Hour: every customer has 3 hearts. One heart is lost every 10 seconds
// of waiting, and the customer leaves (game over) after 30 seconds.
const int MAX_HEARTS = 3;
const unsigned int CUSTOMER_HEART_TIME = 10000;

const unsigned int CUSTOMER_TRANSITION_TIME = 1600;

// Change screens, update timers and handle cake movement and merging.
void advance_screen(game_state &game, unsigned int now);
void update_game(game_state &game, unsigned int now);
void toggle_pause(game_state &game, unsigned int now);
void select_or_move_cake(game_state &game, int row, int column, unsigned int now);

// Return the current order requirements for the user interface.
int current_order_tier(const game_state &game);
int current_order_quantity(const game_state &game);
int current_order_secondary_tier(const game_state &game);
int current_order_secondary_quantity(const game_state &game);
unsigned int current_order_time_left(const game_state &game, unsigned int now);

// After an order is finished the customer leaves, then the next one arrives.
// True once the "Order complete" message is over and the customer has gone.
// Recipe Challenge: how many moves the current order allows in total.
int current_move_limit(const game_state &game);

bool customer_has_left(const game_state &game, unsigned int now);

#endif

// ========= File: src/game_logic.cpp ==========
// src/game_logic.cpp
#include "splashkit.h"
#include "game_state.h"
```

```cpp
#include "game_logic.h"
#include "sound_manager.h"
#include "utilities.h"

static void clear_selection(game_state &game)
{
    game.selected_row = -1;
    game.selected_column = -1;
}

// Story: move through the dialogue, then show the four modes.
// Win or game-over screen: return to the four modes.
void advance_screen(game_state &game, unsigned int now)
{
    (void)now; // kept so existing calls stay the same

    if (game.screen == STORY_SCREEN)
    {
        if (game.story_line < 2)
            game.story_line++;
        else
            show_mode_menu(game);
    }
    else if (game.screen == WON_SCREEN || game.screen == GAME_OVER_SCREEN)
    {
        show_mode_menu(game);
    }
}

item_category current_order_category(const game_state &game)
{
    if (game.order_index >= order_count_for_mode(game.mode)) return NO_ITEM;

    return game.orders[game.order_index].category;
}

int current_order_tier(const game_state &game)
{
    if (game.order_index >= order_count_for_mode(game.mode)) return -1;

    return game.orders[game.order_index].tier;
}

int current_order_quantity(const game_state &game)
{
    if (game.order_index >= order_count_for_mode(game.mode)) return 0;

    return game.orders[game.order_index].quantity;
}

```

```
547 item_category current_order_secondary_category(const game_state &game)
548 {
549     if (game.mode != RECIPE_MODE || game.order_index >= order_count_for_mode(game.mode))
550         return NO_ITEM;
551     return game.orders[game.order_index].secondary_category;
552 }
553
554 int current_order_secondary_tier(const game_state &game)
555 {
556     if (game.mode != RECIPE_MODE || game.order_index >= order_count_for_mode(game.mode))
557         return -1;
558     return game.orders[game.order_index].secondary_tier;
559 }
560
561 int current_order_secondary_quantity(const game_state &game)
562 {
563     if (game.mode != RECIPE_MODE || game.order_index >= order_count_for_mode(game.mode))
564         return 0;
565     return game.orders[game.order_index].secondary_quantity;
566 }
567
568 unsigned int current_order_time_left(const game_state &game, unsigned int now)
569 {
570     if (game.mode != RUSH_MODE || game.screen != PLAYING_SCREEN)
571         return ORDER_TIME_LIMIT;
572     // The timer is paused while the customer is leaving.
573     unsigned int clock = game.order_just_completed ? game.order_completed_at : (game.paused
    ? game.paused_at : now);
574     unsigned int elapsed = clock - game.order_started_at;
575     return (elapsed >= ORDER_TIME_LIMIT) ? 0 : ORDER_TIME_LIMIT - elapsed;
576 }
577
578 int current_move_limit(const game_state &game)
579 {
580     if (game.mode != RECIPE_MODE || game.order_index >= order_count_for_mode(game.mode))
    return 0;
581     return game.orders[game.order_index].move_limit;
582 }
583
584 // Rush Hour: hearts the current customer has left, based on how long they have waited.
585 static int hearts_left(const game_state &game, unsigned int now)
586 {
587     // The clock is paused while the customer is leaving.
588     unsigned int clock = game.order_just_completed ? game.order_completed_at : (game.paused
    ? game.paused_at : now);
589     int hearts_lost = (int)((clock - game.order_started_at) / CUSTOMER_HEART_TIME);
590     if (hearts_lost >= MAX_HEARTS) return 0;
591     return MAX_HEARTS - hearts_lost;
592 }
593
594 bool customer_has_left(const game_state &game, unsigned int now)
```

```
{
    unsigned int clock = game.paused ? game.paused_at : now;
    return game.order_just_completed && clock - game.order_completed_at >=
ORDER_COMPLETE_MESSAGE_TIME;
}

static bool can_make_a_move(const game_state &game)
{
    // An empty square means the player can still generate or move an item.
    for (int row = 0; row < GRID_SIZE; row++)
    {
        for (int col = 0; col < GRID_SIZE; col++)
        {
            if (game.board[row][col].category == NO_ITEM)
                return true;
        }
    }

    // Matching categories and tiers mean that a merge is possible.
    for (int row = 0; row < GRID_SIZE; row++)
    {
        for (int col = 0; col < GRID_SIZE; col++)
        {
            board_item first = game.board[row][col];
            if (first.category == NO_ITEM || first.tier >= MAX_ITEM_TIER)
                continue;
            for (int other_row = 0; other_row < GRID_SIZE; other_row++)
            {
                for (int other_col = 0; other_col < GRID_SIZE; other_col++)
                {
                    if (row == other_row && col == other_col)
                        continue;
                    board_item second = game.board[other_row][other_col];
                    if (first.category == second.category && first.tier == second.tier)
                        return true;
                }
            }
        }
    }
    return false;
}

static bool recipe_food_is_pending(const game_state &game, int row, int column)
{
    return game.recipe_food_pending[row][column];
}

static void update_recipe_food_collection(game_state &game, unsigned int now)
{
    for (int row = 0; row < GRID_SIZE; row++)
```

```
644     {
645         for (int column = 0; column < GRID_SIZE; column++)
646         {
647             if (game.recipe_food_pending[row][column] &&
648                 now - game.recipe_food_delivered_at[row][column] >= ORDER_COMPLETE_MESSA-
GE_TIME)
649             {
650                 game.board[row][column].category = NO_ITEM;
651                 game.board[row][column].tier = 0;
652                 game.recipe_food_pending[row][column] = false;
653             }
654         }
655     }
656 }
657 // Take up to "count" foods of one kind off the board (rows first, then columns).
658 static void remove_foods(game_state &game, item_category category, int tier, int count)
659 {
660     for (int row = 0; row < GRID_SIZE; row++)
661     {
662         for (int col = 0; col < GRID_SIZE; col++)
663         {
664             if (count > 0 && game.board[row][col].category == category && game.board[row]
[col].tier == tier)
665             {
666                 game.board[row][col].category = NO_ITEM;
667                 game.board[row][col].tier = 0;
668                 count--;
669             }
670         }
671     }
672 }
673
674 // The foods that were counted for the current order are taken away
675 static void remove_served_food(game_state &game)
676 {
677     if (game.order_index >= order_count_for_mode(game.mode)) return;
678
679     // Recipe items are removed as soon as each exact requirement is delivered.
680     // Other modes still remove their completed items when the customer leaves.
681     if (game.mode != RECIPE_MODE)
682         remove_foods(game, current_order_category(game), current_order_tier(game),
game.delivered_count);
683 }
684
685 // Generates the requested food type in a randomly chosen empty square.
686 // Returns false if the board has no empty square.
687 bool generate_item(game_state &game, item_category category)
688 {
689     if (game.screen != PLAYING_SCREEN || category == NO_ITEM)
690         return false;
691     int empty_count = 0;
```

```
    for (int row = 0; row < GRID_SIZE; row++)
    {
        for (int col = 0; col < GRID_SIZE; col++)
        {
            if (game.board[row][col].category == NO_ITEM)
                empty_count++;
        }
    }

    if (empty_count == 0)
        return false;

    // Pick one of the empty squares at random.
    int chosen_empty = rnd(empty_count);   // rnd(n) gives 0 to n - 1
    for (int row = 0; row < GRID_SIZE; row++)
    {
        for (int col = 0; col < GRID_SIZE; col++)
        {
            if (game.board[row][col].category == NO_ITEM)
            {
                if (chosen_empty == 0)
                {
                    game.board[row][col].category = category;
                    game.board[row][col].tier = 0;
                    game.last_action = "";
                    play_drop_sound(game);
                    return true;
                }
                chosen_empty--;
            }
        }
    }
    return false;
}

static bool order_is_complete(const game_state &game)
{
    if (game.mode == RECIPE_MODE)
    {
        return game.recipe_first_delivered >= current_order_quantity(game) &&
game.recipe_second_delivered >= current_order_secondary_quantity(game);
    }
    return game.delivered_count >= current_order_quantity(game);
}

static void count_prepared_items_for_order(game_state &game, unsigned int now);
static void complete_order(game_state &game, unsigned int now);

// The next customer arrives with the next order.
static void finish_order_transition(game_state &game, unsigned int now)
```

```
{
    game.order_just_completed = false;
    game.served_food_removed = false;
    game.order_index++;
    game.delivered_count = 0;
    game.recipe_first_delivered = 0;
    game.recipe_second_delivered = 0;
    game.last_action = "";
    game.order_started_at = now;   // Rush Hour: the new order gets a full timer
    game.low_time_alert_played = false;
    game.satisfaction = MAX_HEARTS;   // every new customer starts with full hearts

    if (game.mode == RECIPE_MODE)
        game.moves_left = current_move_limit(game);

    play_question_sound(game);
    count_prepared_items_for_order(game, now);
    complete_order(game, now);
}

static void complete_order(game_state &game, unsigned int now)
{
    if (!order_is_complete(game))
        return;
    play_order_sound(game);
    game.score += 100 * (game.order_index + 1);

    if (game.mode == RUSH_MODE)
        game.score += (int)(current_order_time_left(game, now) / 1000) * 2;

    // The last order goes straight to the win screen.
    if (game.order_index + 1 >= order_count_for_mode(game.mode))
    {
        game.order_index++;
        game.screen = WON_SCREEN;
        return;
    }

    // Keep the hearts this customer had when they were served.
    if (game.mode == RUSH_MODE)
        game.satisfaction = hearts_left(game, now);

    // Otherwise show "Order complete" and let the customer leave first.
    // The order number changes when the next customer arrives.
    game.order_just_completed = true;
    game.served_food_removed = false;
    game.order_completed_at = now;
}

// Count exact matching items that were prepared before this customer arrived.
```

```
static void count_prepared_items_for_order(game_state &game, unsigned int now)
{
    if (game.order_index >= order_count_for_mode(game.mode))
        return;

    bool counted_any = false;
    for (int row = 0; row < GRID_SIZE; row++)
    {
        for (int column = 0; column < GRID_SIZE; column++)
        {
            board_item &item = game.board[row][column];
            if (item.category == NO_ITEM ||
                (game.mode == RECIPE_MODE && recipe_food_is_pending(game, row, column)))
                continue;

            if (game.mode == RECIPE_MODE)
            {
                if (item.category == current_order_category(game) &&
                    item.tier == current_order_tier(game) &&
                    game.recipe_first_delivered < current_order_quantity(game))
                {
                    game.recipe_first_delivered++;
                    game.recipe_food_pending[row][column] = true;
                    game.recipe_food_delivered_at[row][column] = now;
                    counted_any = true;
                }
                else if (item.category == current_order_secondary_category(game) &&
                         item.tier == current_order_secondary_tier(game) &&
                         game.recipe_second_delivered < current_order_second-
ary_quantity(game))
                {
                    game.recipe_second_delivered++;
                    game.recipe_food_pending[row][column] = true;
                    game.recipe_food_delivered_at[row][column] = now;
                    counted_any = true;
                }
            }
            else if (item.category == current_order_category(game) &&
                     item.tier == current_order_tier(game) &&
                     game.delivered_count < current_order_quantity(game))
            {
                game.delivered_count++;
                counted_any = true;
            }
        }
    }

    if (counted_any)
        game.last_action = "Prepared item counted.";
}
```

```
static void record_item_for_order(game_state &game, item_category category, int tier,
                                  int row, int column, unsigned int now)
{
    // Only an exact category and tier advances an order. Recipe deliveries are
    // consumed immediately so a completed ingredient cannot be merged again.
    bool counted = false;
    bool requested_category = false;
    bool requirement_already_met = false;
    int requested_tier = -1;

    if (game.mode == RECIPE_MODE)
    {
        if (category == current_order_category(game))
        {
            requested_category = true;
            requested_tier = current_order_tier(game);
            requirement_already_met = game.recipe_first_delivered >= current_order_quanti-
ty(game);
            if (tier == requested_tier && !requirement_already_met)
            {
                game.recipe_first_delivered++;
                counted = true;
            }
        }
        else if (category == current_order_secondary_category(game))
        {
            requested_category = true;
            requested_tier = current_order_secondary_tier(game);
            requirement_already_met = game.recipe_second_delivered >= current_order_second-
ary_quantity(game);
            if (tier == requested_tier && !requirement_already_met)
            {
                game.recipe_second_delivered++;
                counted = true;
            }
        }
    }
    else if (category == current_order_category(game))
    {
        requested_category = true;
        requested_tier = current_order_tier(game);
        requirement_already_met = game.delivered_count >= current_order_quantity(game);
        if (tier == requested_tier && !requirement_already_met)
        {
            game.delivered_count++;
            counted = true;
        }
    }

    if (counted)
```

```
    {
        game.last_action = "Counted: level " + number_text(tier + 1) + ".";
        if (game.mode == RECIPE_MODE)
        {
            game.recipe_food_pending[row][column] = true;
            game.recipe_food_delivered_at[row][column] = now;
        }
    }
    else if (requirement_already_met)
        game.last_action = "That recipe item is already complete.";
    else if (requested_category)
        game.last_action = "Not counted: needs level " + number_text(requested_tier + 1) +
"."
    else
        game.last_action = "This food is not in the order.";

    complete_order(game, now);
}

static void use_move(game_state &game)
{
    game.moves++;
    if (game.mode == RECIPE_MODE)
        game.moves_left--;
}

// Recipe Challenge: running out of moves before the recipe is finished ends the game.
static void check_recipe_limit(game_state &game)
{
    if (game.mode == RECIPE_MODE &&
        game.screen == PLAYING_SCREEN &&
        !game.order_just_completed &&
        game.moves_left <= 0 &&
        game.order_index < order_count_for_mode(game.mode))
    {
        play_timeout_sound(game);
        game.screen = GAME_OVER_SCREEN;
        game.lost_by_move_limit = true;
    }
}

void select_or_move_cake(game_state &game, int row, int column, unsigned int now)
{
    // The board is locked while the customer is leaving.
    if (game.screen != PLAYING_SCREEN ||
        game.order_just_completed ||
        !is_in_range(row, 0, GRID_SIZE - 1) ||
        !is_in_range(column, 0, GRID_SIZE - 1))
        return;

```

```
    if (game.mode == RECIPE_MODE && recipe_food_is_pending(game, row, column))
    {
        clear_selection(game);
        game.last_action = "This item is being served.";
        return;
    }

    board_item target = game.board[row][column];
    // First click selects an occupied square.
    if (game.selected_row == -1)
    {
        if (target.category != NO_ITEM)
        {
            game.last_action = "";
            game.selected_row = row;
            game.selected_column = column;
        }
        return;
    }

    int from_row = game.selected_row;
    int from_column = game.selected_column;

    if (from_row == row && from_column == column)
    {
        game.last_action = "";
        clear_selection(game);
        return;
    }
    board_item moving = game.board[from_row][from_column];

    // Move an item into an empty square.
    if (target.category == NO_ITEM)
    {
        game.last_action = "";
        game.board[row][column] = moving;
        game.board[from_row][from_column].category = NO_ITEM;
        game.board[from_row][from_column].tier = 0;

        play_drop_sound(game);
        use_move(game);
        check_recipe_limit(game);
        clear_selection(game);
        return;
    }

    // Merge only when both the category and tier match.
    if (target.category == moving.category && target.tier == moving.tier && moving.tier < MAX_ITEM_TIER)
    {
```

```
        int merged_tier = moving.tier + 1;
        game.board[row][column].tier = merged_tier;
        game.board[from_row][from_column].category = NO_ITEM;
        game.board[from_row][from_column].tier = 0;

        play_merge_sound(game);
        game.score += merged_tier * 10;
        use_move(game);
        record_item_for_order(game, target.category, merged_tier, row, column, now);
        check_recipe_limit(game);
        clear_selection(game);
        return;
    }

    // Clicking a different occupied square selects that square instead.
    game.last_action = "";
    game.selected_row = row;
    game.selected_column = column;
}

void toggle_pause(game_state &game, unsigned int now)
{
    if (game.screen != PLAYING_SCREEN)
        return;

    if (!game.paused)
    {
        game.paused = true;
        game.paused_at = now;
        pause_music();
    }
    else
    {
        unsigned int paused_duration = now - game.paused_at;
        for (int row = 0; row < GRID_SIZE; row++)
            for (int column = 0; column < GRID_SIZE; column++)
                if (game.recipe_food_pending[row][column])
                    game.recipe_food_delivered_at[row][column] += paused_duration;
        game.order_started_at += paused_duration;
        if (game.order_just_completed)
            game.order_completed_at += paused_duration;
        game.paused = false;
        game.paused_at = 0;
        apply_music_setting(game);
    }
}

// Called every frame: timers, the customer leaving, and the "no moves left" check.
void update_game(game_state &game, unsigned int now)
{
```

```
1036      if (game.screen != PLAYING_SCREEN || game.paused) return;
1037      if (game.mode == RECIPE_MODE)
1038          update_recipe_food_collection(game, now);
1039      // While the customer is leaving nothing else can happen (the timer is paused).
1040      if (game.order_just_completed)
1041      {
1042          // When the customer leaves, they take the food they ordered with them.
1043          if (customer_has_left(game, now) && !game.served_food_removed)
1044          {
1045              remove_served_food(game);
1046              game.served_food_removed = true;
1047          }
1048
1049          if (now - game.order_completed_at >= CUSTOMER_TRANSITION_TIME)
1050              finish_order_transition(game, now);
1051
1052          return;
1053      }
1054
1055      if (game.mode == RUSH_MODE)
1056      {
1057          // A heart is lost every 10 seconds, with a tick to tell the player.
1058          int hearts = hearts_left(game, now);
1059
1060          if (hearts < game.satisfaction)
1061              play_tick_sound(game);
1062
1063          game.satisfaction = hearts;
1064
1065          // Last 10 seconds: one more warning tick.
1066          if (!game.low_time_alert_played && current_order_time_left(game, now) <= 10000 &&
  current_order_time_left(game, now) > 0)
1067          {
1068              play_tick_sound(game);
1069              game.low_time_alert_played = true;
1070          }
1071
1072          // After 30 seconds the customer leaves and the game is over.
1073          if (current_order_time_left(game, now) == 0)
1074          {
1075              play_timeout_sound(game);
1076              game.screen = GAME_OVER_SCREEN;
1077              game.lost_by_timer = true;
1078              return;
1079          }
1080      }
1081
1082      if (game.screen == PLAYING_SCREEN && !can_make_a_move(game))
1083          game.screen = GAME_OVER_SCREEN;
1084  }
```

```
1085
1086 // ========= File: src/score_manager.h ==========
1087 //src/score_manager.h
1088 #ifndef SCORE_MANAGER_H
1089 #define SCORE_MANAGER_H
1090
1091 #include "game_state.h"
1092
1093 int high_score_for_mode(game_mode mode);
1094 bool record_high_score(game_mode mode, int score);
1095
1096 #endif
1097
1098 // ========= File: src/score_manager.cpp ==========
1099 //src/score_manager.cpp
1100 #include "splashkit.h"
1101 #include "score_manager.h"
1102 #include <fstream>
1103
1104 static const int MODE_COUNT = 3;
1105 static int high_scores[MODE_COUNT] = {0, 0, 0};
1106 static bool scores_loaded = false;
1107
1108 static int mode_index(game_mode mode)
1109 {
1110     // Beginner has a fixed learning recipe and does not keep a high mark.
1111     switch (mode)
1112     {
1113         case CLASSIC_MODE: return 0;
1114         case RUSH_MODE: return 1;
1115         case RECIPE_MODE: return 2;
1116         default: return -1;
1117     }
1118 }
1119
1120 static void load_high_scores()
1121 {
1122     if (scores_loaded) return;
1123
1124     std::ifstream input("high_scores.txt");
1125     for (int i = 0; i < MODE_COUNT; i++)
1126     {
1127         int score = 0;
1128         if (input >> score && score >= 0)
1129             high_scores[i] = score;
1130     }
1131     scores_loaded = true;
1132 }
1133
1134 static void save_high_scores()
```

```
{
    std::ofstream output("high_scores.txt", std::ios::trunc);
    if (!output)
    {
        write_line("Could not save high scores. Run the game from its project folder.");
        return;
    }

    for (int i = 0; i < MODE_COUNT; i++)
        output << high_scores[i] << '\n';
}

int high_score_for_mode(game_mode mode)
{
    load_high_scores();
    int index = mode_index(mode);
    return (index < 0) ? 0 : high_scores[index];
}

bool record_high_score(game_mode mode, int score)
{
    load_high_scores();
    int index = mode_index(mode);
    if (index < 0 || score <= high_scores[index]) return false;

    high_scores[index] = score;
    save_high_scores();
    return true;
}

// ========= File: src/sound_manager.h ==========
// src/sound_manager.h
#ifndef SOUND_MANAGER_H
#define SOUND_MANAGER_H

#include "splashkit.h"
#include "game_state.h"

void load_game_sounds();
void play_story_voice(const game_state &game, int story_line);
void stop_story_voice();
void load_background_music();
void play_background_music();
// Pause or resume the music to match the player's music setting.
void apply_music_setting(const game_state &game);
void play_merge_sound(const game_state &game);
void play_order_sound(const game_state &game);
void play_timeout_sound(const game_state &game);
void play_drop_sound(const game_state &game);
void play_click_sound(const game_state &game);
```

```
1185 void play_question_sound(const game_state &game);
1186 void play_tick_sound(const game_state &game);
1187
1188 #endif
1189
1190 // ========= File: src/sound_manager.cpp ==========
1191 // src/sound_manager.cpp
1192 #include "splashkit.h"
1193 #include "sound_manager.h"
1194
1195 void load_game_sounds()
1196 {
1197     // Load every short effect once before the game loop starts.
1198     load_sound_effect("merge", "Resources/audio/merge_pop.wav");
1199     load_sound_effect("order", "Resources/audio/confirmation.ogg");
1200     load_sound_effect("timeout", "Resources/audio/error.ogg");
1201     load_sound_effect("drop", "Resources/audio/drop.ogg");
1202     load_sound_effect("click", "Resources/audio/click.ogg");
1203     load_sound_effect("question", "Resources/audio/question.ogg");
1204     load_sound_effect("tick", "Resources/audio/tick.ogg");
1205
1206     // Story narration for Mia and the festival guest.
1207     load_sound_effect("story_mia_intro", "Resources/audio/story_mia_intro.mp3");
1208     load_sound_effect("story_festival_guest", "Resources/audio/story_festival_guest.mp3");
1209     load_sound_effect("story_mia_invitation", "Resources/audio/story_mia_invitation.mp3");
1210 }
1211
1212 void stop_story_voice()
1213 {
1214     stop_sound_effect("story_mia_intro");
1215     stop_sound_effect("story_festival_guest");
1216     stop_sound_effect("story_mia_invitation");
1217 }
1218
1219 void play_story_voice(const game_state &game, int story_line)
1220 {
1221     stop_story_voice();
1222     if (!game.sound_effects_on) return;
1223
1224     if (story_line == 0)
1225         play_sound_effect("story_mia_intro", 0.85);
1226     else if (story_line == 1)
1227         play_sound_effect("story_festival_guest", 0.85);
1228     else if (story_line == 2)
1229         play_sound_effect("story_mia_invitation", 0.85);
1230 }
1231
1232 // Play one effect, but only when the player has sound effects switched on.
1233 static void play_effect(const game_state &game, const string &name, double volume)
1234 {
```

```
    if (game.sound_effects_on)
        play_sound_effect(name, volume);
}

// Keep effects slightly below full volume so they do not overpower the game.
void play_merge_sound(const game_state &game)
{
    play_effect(game, "merge", 0.75);
}

void play_order_sound(const game_state &game)
{
    play_effect(game, "order", 0.7);
}

void play_timeout_sound(const game_state &game)
{
    play_effect(game, "timeout", 0.55);
}

void play_drop_sound(const game_state &game)
{
    play_effect(game, "drop", 0.45);
}

void play_click_sound(const game_state &game)
{
    play_effect(game, "click", 0.45);
}

void play_question_sound(const game_state &game)
{
    play_effect(game, "question", 0.45);
}

void play_tick_sound(const game_state &game)
{
    play_effect(game, "tick", 0.35);
}

// Load and play a soft bakery theme throughout the game.
static music background_music;
void load_background_music()
{
    background_music = load_music("bakery_bgm", "Resources/audio/bakery_bgm.mp3");
}

void play_background_music()
{
    play_music(background_music, -1, 0.20);
```

```
}

// The music keeps its place in the song while it is paused.
void apply_music_setting(const game_state &game)
{
    if (game.music_on)
        resume_music();
    else
        pause_music();
}

// ========= File: src/input.h ==========
// src/input.h
#ifndef INPUT_H
#define INPUT_H

#include "splashkit.h"
#include "game_state.h"

// Route a mouse click to a menu button or a board cell.
void handle_mouse_click(game_state &game, double mouse_x, double mouse_y, unsigned int now);

// Handle the keyboard controls for the current screen.
void handle_key_presses(game_state &game, unsigned int now);

#endif

// ========= File: src/input.cpp ==========
// src/input.cpp
#include "splashkit.h"
#include "input.h"
#include "game_logic.h"
#include "game_layout.h"
#include "sound_manager.h"
#include "utilities.h"

// Stop the current narration when the player advances, then speak the next subtitle.
static void advance_story_with_voice(game_state &game, unsigned int now)
{
    stop_story_voice();
    advance_screen(game, now);
    if (game.screen == STORY_SCREEN)
        play_story_voice(game, game.story_line);
}

// Return which generator was clicked, or NO_ITEM if the click missed them all.
static item_category generator_at(double mouse_x, double mouse_y)
{
    if (point_in_rectangle(mouse_x, mouse_y, DOUGH_BUTTON_X, GENERATOR_Y, GENERATOR_WIDTH,
GENERATOR_HEIGHT))
```

```
1334        return DOUGH;
1335
1336    if (point_in_rectangle(mouse_x, mouse_y, TART_BUTTON_X, GENERATOR_Y, GENERATOR_WIDTH,
    GENERATOR_HEIGHT))
1337        return TART;
1338
1339    if (point_in_rectangle(mouse_x, mouse_y, CAKE_BUTTON_X, GENERATOR_Y, GENERATOR_WIDTH,
    GENERATOR_HEIGHT))
1340        return CAKE;
1341
1342    return NO_ITEM;
1343 }
1344
1345 // Handle a click inside the settings panel. The two switches work independently,
1346 // and clicks outside the panel do nothing while it is open.
1347 static void handle_settings_click(game_state &game, double mouse_x, double mouse_y)
1348 {
1349     if (point_in_rectangle(mouse_x, mouse_y, SETTINGS_TOGGLE_X, SETTINGS_MUSIC_Y,
1350             SETTINGS_TOGGLE_WIDTH, SETTINGS_TOGGLE_HEIGHT))
1351     {
1352         toggle_music(game);
1353         apply_music_setting(game);
1354         play_click_sound(game);
1355     }
1356     else if (point_in_rectangle(mouse_x, mouse_y, SETTINGS_TOGGLE_X, SETTINGS_EFFECTS_Y,
1357             SETTINGS_TOGGLE_WIDTH, SETTINGS_TOGGLE_HEIGHT))
1358     {
1359         toggle_sound_effects(game);
1360         play_click_sound(game);   // only heard when effects have just been switched ON
1361     }
1362     else if (point_in_rectangle(mouse_x, mouse_y, SETTINGS_CLOSE_X, SETTINGS_CLOSE_Y,
1363             SETTINGS_CLOSE_WIDTH, SETTINGS_CLOSE_HEIGHT))
1364     {
1365         play_click_sound(game);
1366         close_settings(game);
1367     }
1368 }
1369
1370 // Handle a click on the four mode cards. Choosing a mode starts gameplay at once.
1371 static void handle_menu_click(game_state &game, double mouse_x, double mouse_y, unsigned int
    now)
1372 {
1373     if (game.settings_open)
1374     {
1375         handle_settings_click(game, mouse_x, mouse_y);
1376         return;
1377     }
1378
1379     if (point_in_rectangle(mouse_x, mouse_y, SETTINGS_BUTTON_X, SETTINGS_BUTTON_Y,
1380             SETTINGS_BUTTON_WIDTH, SETTINGS_BUTTON_HEIGHT))
1381     {
```

```
1382            play_click_sound(game);
1383            open_settings(game);
1384            return;
1385        }
1386
1387        game_mode chosen = BEGINNER_MODE;
1388        bool clicked_card = true;
1389
1390        if (point_in_rectangle(mouse_x, mouse_y, MODE_LEFT_X, MODE_TOP_Y, MODE_CARD_WIDTH,
    MODE_CARD_HEIGHT))
1391            chosen = BEGINNER_MODE;
1392        else if (point_in_rectangle(mouse_x, mouse_y, MODE_RIGHT_X, MODE_TOP_Y, MODE_CARD_WIDTH,
    MODE_CARD_HEIGHT))
1393            chosen = CLASSIC_MODE;
1394        else if (point_in_rectangle(mouse_x, mouse_y, MODE_LEFT_X, MODE_BOTTOM_Y,
    MODE_CARD_WIDTH, MODE_CARD_HEIGHT))
1395            chosen = RUSH_MODE;
1396        else if (point_in_rectangle(mouse_x, mouse_y, MODE_RIGHT_X, MODE_BOTTOM_Y,
    MODE_CARD_WIDTH, MODE_CARD_HEIGHT))
1397            chosen = RECIPE_MODE;
1398        else
1399            clicked_card = false;
1400
1401        if (clicked_card)
1402        {
1403            play_click_sound(game);
1404            choose_mode(game, chosen, now);
1405        }
1406    }
1407
1408    // Handle a click on the 4 x 4 board: select, move or merge.
1409    static void handle_board_click(game_state &game, double mouse_x, double mouse_y, unsigned
    int now)
1410    {
1411        if (!point_in_rectangle(mouse_x, mouse_y, BOARD_X, BOARD_Y, BOARD_SIZE, BOARD_SIZE))
1412            return;
1413
1414        int column = (int)(mouse_x - BOARD_X) / (CELL_SIZE + CELL_GAP);
1415        int row = (int)(mouse_y - BOARD_Y) / (CELL_SIZE + CELL_GAP);
1416        int in_cell_x = (int)(mouse_x - BOARD_X) % (CELL_SIZE + CELL_GAP);
1417        int in_cell_y = (int)(mouse_y - BOARD_Y) % (CELL_SIZE + CELL_GAP);
1418
1419        // Ignore the small gaps between board squares.
1420        if (in_cell_x < CELL_SIZE && in_cell_y < CELL_SIZE)
1421        {
1422            select_or_move_cake(game, row, column, now);
1423        }
1424    }
1425
1426    void handle_mouse_click(game_state &game, double mouse_x, double mouse_y, unsigned int now)
1427    {
```

```
1428      if (game.screen == MODE_MENU)
1429      {
1430          handle_menu_click(game, mouse_x, mouse_y, now);
1431          return;
1432      }
1433
1434      if (game.screen == STORY_SCREEN)
1435      {
1436          play_click_sound(game);
1437          advance_story_with_voice(game, now);
1438          return;
1439      }
1440
1441      if (game.screen == WON_SCREEN || game.screen == GAME_OVER_SCREEN)
1442      {
1443          if (point_in_rectangle(mouse_x, mouse_y, END_REPLAY_BUTTON_X, END_BUTTON_Y,
1444                  END_BUTTON_WIDTH, END_BUTTON_HEIGHT))
1445          {
1446              play_click_sound(game);
1447              choose_mode(game, game.mode, now);
1448          }
1449          else if (point_in_rectangle(mouse_x, mouse_y, END_MENU_BUTTON_X, END_BUTTON_Y,
1450                  END_BUTTON_WIDTH, END_BUTTON_HEIGHT))
1451          {
1452              play_click_sound(game);
1453              show_mode_menu(game);
1454          }
1455          else
1456          {
1457              play_click_sound(game);
1458              advance_screen(game, now);
1459          }
1460          return;
1461      }
1462
1463      if (game.screen != PLAYING_SCREEN)
1464          return;
1465
1466      // Pause/Resume remains available while the game is paused.
1467      if (point_in_rectangle(mouse_x, mouse_y, PAUSE_BUTTON_X, PAUSE_BUTTON_Y,
1468              PAUSE_BUTTON_WIDTH, PAUSE_BUTTON_HEIGHT))
1469      {
1470          play_click_sound(game);
1471          toggle_pause(game, now);
1472          return;
1473      }
1474
1475      // The BACK button returns to mode selection without closing the game.
1476      if (point_in_rectangle(mouse_x, mouse_y, BACK_X, BACK_Y, BACK_WIDTH, BACK_HEIGHT))
1477      {
```

```
1478            play_click_sound(game);
1479            show_mode_menu(game);
1480            return;
1481        }
1482
1483        if (game.paused)
1484            return;
1485
1486        // A generator button puts its food in a random empty square straight away.
1487        item_category generator = generator_at(mouse_x, mouse_y);
1488
1489        if (generator != NO_ITEM)
1490        {
1491            if (generate_item(game, generator))
1492                play_click_sound(game);
1493            else
1494                play_timeout_sound(game); // the board is full
1495            return;
1496        }
1497
1498        handle_board_click(game, mouse_x, mouse_y, now);
1499    }
1500
1501    // Keyboard controls: 1-4 pick a mode, SPACE continues, R goes back to the modes.
1502    void handle_key_presses(game_state &game, unsigned int now)
1503    {
1504        if (game.screen == MODE_MENU)
1505        {
1506            // While the settings panel is open only ESC works.
1507            if (game.settings_open)
1508            {
1509                if (key_typed(ESCAPE_KEY))
1510                    close_settings(game);
1511                return;
1512            }
1513
1514            if (key_typed(NUM_1_KEY))
1515            {
1516                play_click_sound(game);
1517                choose_mode(game, BEGINNER_MODE, now);
1518            }
1519            else if (key_typed(NUM_2_KEY))
1520            {
1521                play_click_sound(game);
1522                choose_mode(game, CLASSIC_MODE, now);
1523            }
1524            else if (key_typed(NUM_3_KEY))
1525            {
1526                play_click_sound(game);
1527                choose_mode(game, RUSH_MODE, now);
```

```
        }
        else if (key_typed(NUM_4_KEY))
        {
            play_click_sound(game);
            choose_mode(game, RECIPE_MODE, now);
        }
        return;
    }

    if (game.screen == PLAYING_SCREEN && (key_typed(P_KEY) || key_typed(ESCAPE_KEY)))
    {
        play_click_sound(game);
        toggle_pause(game, now);
        return;
    }

    if (game.paused)
    {
        if (key_typed(R_KEY))
            show_mode_menu(game);
        return;
    }

    if (key_typed(SPACE_KEY))
    {
        if (game.screen == STORY_SCREEN)
            advance_story_with_voice(game, now);
        else
            advance_screen(game, now);
    }

    // R returns to the four modes (never back to the story).
    if (key_typed(R_KEY) && game.screen != STORY_SCREEN)
    {
        show_mode_menu(game);
    }
}

// ========= File: src/game_view.h ==========
// src/game_view.h
#ifndef GAME_VIEW_H
#define GAME_VIEW_H

#include "splashkit.h"
#include "game_state.h"

void draw_game(const game_state &game, unsigned int now);
void load_game_art();

#endif
```

```
1578
1579 // ========= File: src/game_view.cpp ==========
1580 // src/game_view.cpp
1581 #include "splashkit.h"
1582 #include "game_view.h"
1583 #include "sound_manager.h"
1584 #include "score_manager.h"
1585 #include "game_logic.h"
1586 #include "game_layout.h"
1587 #include "utilities.h"
1588
1589 // Keep the colour creation in one place so the rest of the file stays readable.
1590 static color rgb(int r, int g, int b)
1591 {
1592     return rgb_color(r, g, b);
1593 }
1594
1595 static const string UI_FONT_NAME = "sweetstack_ui";
1596 static const int UI_FONT_SIZE = 14;
1597 static bool ui_font_loaded = false;
1598
1599 static void draw_game_text(const string &text, const color &colour, double x, double y)
1600 {
1601     if (ui_font_loaded)
1602         draw_text(text, colour, UI_FONT_NAME, UI_FONT_SIZE, x, y);
1603     else
1604         draw_text(text, colour, x, y);
1605 }
1606
1607 static int centered_ui_text_x(const string &text, int box_x, int box_width)
1608 {
1609     if (ui_font_loaded)
1610         return box_x + (box_width - text_width(text, UI_FONT_NAME, UI_FONT_SIZE)) / 2;
1611     return centered_text_x(text, box_x, box_width);
1612 }
1613
1614 // Return a readable name for each food chain.
1615 static string item_name(item_category category)
1616 {
1617     if (category == DOUGH)
1618     {
1619         return "Dough";
1620     }
1621     else if (category == TART)
1622     {
1623         return "Tart";
1624     }
1625     else if (category == CAKE)
1626     {
1627         return "Cake";
```

```
    }

    return "food";
}

// Load one picture and tell the player in the terminal if the file was not found.
static void load_picture(const string &name, const string &path)
{
    load_bitmap(name, path);

    if (!has_bitmap(name))
    {
        write_line("Could not load picture: " + path);
        write_line("Run the game from inside the SweetStack folder.");
    }
}

// Load the background, food pictures, shop owner, and customer portraits.
void load_game_art()
{
    load_font(UI_FONT_NAME, "Resources/fonts/Coiny-Regular.ttf");
    ui_font_loaded = has_font(UI_FONT_NAME);
    if (!ui_font_loaded)
        write_line("Could not load font: Resources/fonts/Coiny-Regular.ttf");

    load_picture("bakery_background", "Resources/images/background.png");
    load_picture("shop_owner", "Resources/images/shop_owner_story.png");
    for (int row = 0; row < 3; row++)
    {
        for (int column = 0; column < 4; column++)
        {
            string name = "food_" + number_text(row) + "_" + number_text(column);
            load_picture(name, "Resources/images/food/" + name + ".png");
        }
    }

    // generator_0 = dough mixer, generator_1 = tart oven, generator_2 = cake mixer.
    for (int i = 0; i < 3; i++)
    {
        string name = "generator_" + number_text(i);
        load_picture(name, "Resources/images/food/" + name + ".png");
    }

    // The customer portraits used on the play screen.
    for (int i = 1; i <= 5; i++)
    {
        string file = "customer_0" + number_text(i);
        load_picture("customer_" + number_text(i), "Resources/images/customers/" + file +
".png");
    }
```

```
    load_picture("customer_story_1", "Resources/images/customers/customer_01_story.png");
}

// Convert the current mode into a label for the screen.
static string mode_name(game_mode mode)
{
    if (mode == BEGINNER_MODE)
    {
        return "BEGINNER";
    }
    else if (mode == CLASSIC_MODE)
    {
        return "CLASSIC BAKERY";
    }
    else if (mode == RUSH_MODE)
    {
        return "RUSH HOUR";
    }

    return "RECIPE CHALLENGE";
}

// Draw a food item from the atlas.
// Each row represents a food category and each column represents its tier.
static void draw_item(item_category category, int tier, int x, int y)
{
    if (category == NO_ITEM)
    {
        return;
    }

    int atlas_row = 0;
    if (category == DOUGH)
    {
        atlas_row = 0;
    }
    else if (category == TART)
    {
        atlas_row = 1;
    }
    else if (category == CAKE)
    {
        atlas_row = 2;
    }

    // Higher tiers reuse the final picture.
    int atlas_column = tier;
    if (atlas_column > 3)
    {
        atlas_column = 3;
```

```
    }

    if (atlas_column < 0)
    {
        atlas_column = 0;
    }

    string name = "food_" + number_text(atlas_row) + "_" + number_text(atlas_column);
    draw_bitmap(name, x + FOOD_MARGIN, y + FOOD_MARGIN);
}

// Draw a customer or the shop owner at a chosen position.
static void draw_character(const string &name, int x, int y)
{
    if (has_bitmap(name))
    {
        draw_bitmap(name, x, y);
    }
    else
    {
        // The picture is missing: draw a plain marker so the problem is easy to see.
        fill_circle(rgb(211, 93, 127), x + 69, y + 63, 40);
        draw_game_text("?", rgb(255, 255, 255), x + 65, y + 55);
    }
}

// Draw the title bar used by each screen.
static void draw_header(const string &subtitle)
{
    fill_rectangle(rgb(255, 232, 221), 0, 0, WINDOW_WIDTH, 112);
    draw_game_text("SWEET STACK", rgb(121, 69, 84), 54, 22);
    draw_game_text(subtitle, rgb(146, 109, 116), 56, 62);
}

// Draw one mode choice on the mode-selection screen.
static void mode_card(int x, int y, const char *title, const char *line_a, const char
*line_b, color accent)
{
    fill_rectangle(rgb(255, 255, 255), x, y, MODE_CARD_WIDTH, MODE_CARD_HEIGHT);
    draw_rectangle(accent, x, y, MODE_CARD_WIDTH, MODE_CARD_HEIGHT);
    draw_game_text(title, accent, x + 20, y + 16);
    draw_game_text(line_a, rgb(91, 78, 82), x + 20, y + 55);
    draw_game_text(line_b, rgb(126, 116, 116), x + 20, y + 86);
}

// Draw all four game-mode choices.
// Draw one ON/OFF button for the settings panel.
static void draw_toggle(int y, bool is_on)
{
    color colour = rgb(176, 166, 166);
```

```
1776      string label = "OFF";
1777
1778      if (is_on)
1779      {
1780          colour = rgb(105, 157, 126);
1781          label = "ON";
1782      }
1783
1784      fill_rectangle(colour, SETTINGS_TOGGLE_X, y, SETTINGS_TOGGLE_WIDTH,
SETTINGS_TOGGLE_HEIGHT);
1785      draw_game_text(label, rgb(255, 255, 255),
1786          centered_ui_text_x(label, SETTINGS_TOGGLE_X, SETTINGS_TOGGLE_WIDTH), y + 12);
1787  }
1788
1789  // Draw the settings panel on top of the mode-selection screen.
1790  static void draw_settings_panel(const game_state &game)
1791  {
1792      int label_x = SETTINGS_BOX_X + 30;
1793
1794      // A darker copy offset behind the panel gives it a shadow.
1795      fill_rectangle(rgb(121, 69, 84), SETTINGS_BOX_X + 6, SETTINGS_BOX_Y + 6,
1796          SETTINGS_BOX_WIDTH, SETTINGS_BOX_HEIGHT);
1797      fill_rectangle(rgb(255, 255, 255), SETTINGS_BOX_X, SETTINGS_BOX_Y,
1798          SETTINGS_BOX_WIDTH, SETTINGS_BOX_HEIGHT);
1799      draw_rectangle(rgb(121, 69, 84), SETTINGS_BOX_X, SETTINGS_BOX_Y,
1800          SETTINGS_BOX_WIDTH, SETTINGS_BOX_HEIGHT);
1801
1802      draw_game_text("SETTINGS", rgb(121, 69, 84),
1803          centered_ui_text_x("SETTINGS", SETTINGS_BOX_X, SETTINGS_BOX_WIDTH), SETTINGS_BOX_Y +
20);
1804
1805      draw_game_text("Background music", rgb(91, 78, 82), label_x, SETTINGS_MUSIC_Y + 12);
1806      draw_toggle(SETTINGS_MUSIC_Y, game.music_on);
1807
1808      draw_game_text("Sound effects", rgb(91, 78, 82), label_x, SETTINGS_EFFECTS_Y + 12);
1809      draw_toggle(SETTINGS_EFFECTS_Y, game.sound_effects_on);
1810
1811      draw_game_text("Changes stay until you change them.", rgb(126, 116, 116),
1812          centered_ui_text_x("Changes stay until you change them.", SETTINGS_BOX_X,
SETTINGS_BOX_WIDTH),
1813          SETTINGS_BOX_Y + 200);
1814
1815      fill_rectangle(rgb(121, 69, 84), SETTINGS_CLOSE_X, SETTINGS_CLOSE_Y,
1816          SETTINGS_CLOSE_WIDTH, SETTINGS_CLOSE_HEIGHT);
1817      draw_game_text("CLOSE", rgb(255, 255, 255),
1818          centered_ui_text_x("CLOSE", SETTINGS_CLOSE_X, SETTINGS_CLOSE_WIDTH),
SETTINGS_CLOSE_Y + 14);
1819  }
1820
1821  static void draw_menu(const game_state &game)
1822  {
```

```
    draw_header("Choose how you want to bake");

    fill_rectangle(rgb(121, 69, 84), SETTINGS_BUTTON_X, SETTINGS_BUTTON_Y,
        SETTINGS_BUTTON_WIDTH, SETTINGS_BUTTON_HEIGHT);
    draw_game_text("SETTINGS", rgb(255, 255, 255),
        centered_ui_text_x("SETTINGS", SETTINGS_BUTTON_X, SETTINGS_BUTTON_WIDTH),
SETTINGS_BUTTON_Y + 10);

    mode_card(MODE_LEFT_X, MODE_TOP_Y, "1  BEGINNER",
        "Learn as you play", "One-item orders; no timer", rgb(105, 157, 126));

    mode_card(MODE_RIGHT_X, MODE_TOP_Y, "2  CLASSIC BAKERY",
        "Five random customers", "Orders ask for 1 or 2 items", rgb(116, 139, 184));

    mode_card(MODE_LEFT_X, MODE_BOTTOM_Y, "3  RUSH HOUR",
        "30s per order", "Random; 10s per heart fades", rgb(207, 127, 99));

    mode_card(MODE_RIGHT_X, MODE_BOTTOM_Y, "4  RECIPE CHALLENGE",
        "Limited moves for each order", "Random two-food recipes", rgb(165, 117, 179));

    fill_rectangle(rgb(121, 69, 84), 350, 535, 260, 56);
    draw_game_text("CLICK A MODE (OR PRESS 1-4)", rgb(255, 255, 255), 372, 554);

    if (game.settings_open)
        draw_settings_panel(game);
}

// Draw the story scene with the shop owner and the customer.
static void draw_story(const game_state &game)
{
    draw_header("A little story before baking");

    if (game.story_line == 1)
    {
        draw_character("customer_story_1", 52, 212);
    }
    else
    {
        draw_character("shop_owner", 52, 212);
    }

    // Name plate under the speaker, so the player knows who is talking.
    string speaker = "MIA - SHOP OWNER";
    if (game.story_line == 1)
        speaker = "FESTIVAL GUEST";

    draw_game_text(speaker, rgb(121, 69, 84),
        centered_ui_text_x(speaker, STORY_NAME_X, STORY_NAME_WIDTH), STORY_NAME_Y + 5);

    fill_rectangle(rgb(255, 255, 255), 295, 232, 570, 184);
```

```
    draw_rectangle(rgb(231, 207, 207), 295, 232, 570, 184);
    draw_game_text("Mia's Moonlight Bakery", rgb(121, 69, 84), 325, 253);

    if (game.story_line == 0)
    {
        draw_game_text("Mia: Hi, I'm Mia, the baker who runs this shop.", rgb(91, 78, 82), 325, 289);
        draw_game_text("The Moonlight Festival is tonight!", rgb(91, 78, 82), 325, 325);
        draw_game_text("Our bakery has just reopened.", rgb(91, 78, 82), 325, 358);
    }
    else if (game.story_line == 1)
    {
        draw_game_text("Customer: We need treats for the festival.", rgb(91, 78, 82), 325, 299);
        draw_game_text("Can you help us prepare the orders?", rgb(91, 78, 82), 325, 330);
    }
    else
    {
        draw_game_text("Mia: Choose your challenge and let's bake!", rgb(91, 78, 82), 325, 299);
        draw_game_text("Every successful recipe helps the festival.", rgb(91, 78, 82), 325, 330);
    }

    fill_rectangle(rgb(121, 69, 84), 390, 482, 180, 54);
    draw_game_text("CONTINUE", rgb(255, 255, 255), 435, 500);
    draw_game_text("Click or press SPACE", rgb(255, 255, 255), 395, 555);
}

// Draw one generator button and a small preview of the food it creates.
static void draw_generator_button(const char *label, item_category category, int x)
{
    fill_rectangle(rgb(255, 242, 235), x, GENERATOR_Y, GENERATOR_WIDTH, GENERATOR_HEIGHT);
    draw_rectangle(rgb(201, 157, 145), x, GENERATOR_Y, GENERATOR_WIDTH, GENERATOR_HEIGHT);

    // The generator atlas has three square sprites in one horizontal row.
    int sprite_column = 0;
    if (category == TART)
    {
        sprite_column = 1;
    }
    else if (category == CAKE)
    {
        sprite_column = 2;
    }

    draw_bitmap("generator_" + number_text(sprite_column), x + 4, GENERATOR_Y + 4);
    draw_game_text(label, rgb(121, 69, 84), x + 58, GENERATOR_Y + 19);
}

// Draw the three generators below the 4 by 4 board.
```

```
1919 static void draw_generators()
1920 {
1921     draw_generator_button("DOUGH", DOUGH, DOUGH_BUTTON_X);
1922     draw_generator_button("TART", TART, TART_BUTTON_X);
1923     draw_generator_button("CAKE", CAKE, CAKE_BUTTON_X);
1924 }
1925
1926 // Beginner mode arrows point to a useful matching pair, or the needed generator.
1927 static int draw_beginner_guidance(const game_state &game)
1928 {
1929     if (game.mode != BEGINNER_MODE || game.order_just_completed || game.order_index >=
    order_count_for_mode(game.mode))
1930         return 3;
1931
1932     item_category needed = current_order_category(game);
1933     int required_tier = current_order_tier(game);
1934     int first_row = -1, first_col = -1, second_row = -1, second_col = -1;
1935
1936     if (game.selected_row >= 0 && game.selected_column >= 0)
1937     {
1938         board_item selected = game.board[game.selected_row][game.selected_column];
1939         if (selected.category == needed && selected.tier < required_tier)
1940         {
1941             for (int row = 0; row < GRID_SIZE; row++)
1942             {
1943                 for (int col = 0; col < GRID_SIZE; col++)
1944                 {
1945                     board_item item = game.board[row][col];
1946                     if ((row != game.selected_row || col != game.selected_column) &&
1947                         item.category == selected.category && item.tier == selected.tier)
1948                     {
1949                         second_row = row;
1950                         second_col = col;
1951                         break;
1952                     }
1953                 }
1954                 if (second_row >= 0) break;
1955             }
1956             if (second_row >= 0)
1957             {
1958                 int x = BOARD_X + second_col * (CELL_SIZE + CELL_GAP) + CELL_SIZE / 2;
1959                 int y = BOARD_Y + second_row * (CELL_SIZE + CELL_GAP);
1960                 draw_line(rgb(221, 142, 66), x, y - 28, x, y - 13);
1961                 fill_triangle(rgb(221, 142, 66), x - 8, y - 17, x + 8, y - 17, x, y - 4);
1962                 return 2;
1963             }
1964         }
1965     }
1966
1967     for (int row = 0; row < GRID_SIZE; row++)
```

```
1968    {
1969        for (int col = 0; col < GRID_SIZE; col++)
1970        {
1971            board_item first = game.board[row][col];
1972            if (first.category != needed || first.tier >= required_tier) continue;
1973            for (int other_row = row; other_row < GRID_SIZE; other_row++)
1974            {
1975                int start_col = (other_row == row) ? col + 1 : 0;
1976                for (int other_col = start_col; other_col < GRID_SIZE; other_col++)
1977                {
1978                    board_item second = game.board[other_row][other_col];
1979                    if (second.category == first.category && second.tier == first.tier)
1980                    {
1981                        first_row = row; first_col = col;
1982                        second_row = other_row; second_col = other_col;
1983                        break;
1984                    }
1985                }
1986                if (second_row >= 0) break;
1987            }
1988            if (second_row >= 0) break;
1989        }
1990        if (second_row >= 0) break;
1991    }
1992
1993    if (second_row >= 0)
1994    {
1995        int first_x = BOARD_X + first_col * (CELL_SIZE + CELL_GAP) + CELL_SIZE / 2;
1996        int first_y = BOARD_Y + first_row * (CELL_SIZE + CELL_GAP);
1997        int second_x = BOARD_X + second_col * (CELL_SIZE + CELL_GAP) + CELL_SIZE / 2;
1998        int second_y = BOARD_Y + second_row * (CELL_SIZE + CELL_GAP);
1999        draw_line(rgb(221, 142, 66), first_x, first_y - 28, first_x, first_y - 13);
2000        fill_triangle(rgb(221, 142, 66), first_x - 8, first_y - 17, first_x + 8, first_y -
    17, first_x, first_y - 4);
2001        draw_line(rgb(221, 142, 66), second_x, second_y - 28, second_x, second_y - 13);
2002        fill_triangle(rgb(221, 142, 66), second_x - 8, second_y - 17, second_x + 8, second_y
    - 17, second_x, second_y - 4);
2003        return 1;
2004    }
2005
2006    bool has_empty = false;
2007    for (int row = 0; row < GRID_SIZE; row++)
2008        for (int col = 0; col < GRID_SIZE; col++)
2009            if (game.board[row][col].category == NO_ITEM) has_empty = true;
2010    if (!has_empty) return 3;
2011
2012    int button_x = (needed == DOUGH) ? DOUGH_BUTTON_X : (needed == TART) ? TART_BUTTON_X :
    CAKE_BUTTON_X;
2013    int x = button_x + GENERATOR_WIDTH / 2;
2014    int y = GENERATOR_Y;
2015    draw_line(rgb(221, 142, 66), x, y - 28, x, y - 13);
```

```
    fill_triangle(rgb(221, 142, 66), x - 8, y - 17, x + 8, y - 17, x, y - 4);
    return 0;
}

// Draw the customer on the left and their order on the right.
static void draw_customer_and_order(const game_state &game, unsigned int now)
{
    bool leaving = customer_has_left(game, now);
    int text_x = ORDER_BOX_X + 16;

    // CUSTOMER box and portrait (the portrait is missing once the customer has left).
    fill_rectangle(rgb(255, 255, 255), RIGHT_X, CUSTOMER_BOX_Y, CUSTOMER_BOX_WIDTH,
CUSTOMER_BOX_HEIGHT);
    draw_rectangle(rgb(231, 207, 207), RIGHT_X, CUSTOMER_BOX_Y, CUSTOMER_BOX_WIDTH,
CUSTOMER_BOX_HEIGHT);

    if (!leaving)
    {
        int customer_number = (game.order_index % 5) + 1;
        draw_character("customer_" + number_text(customer_number), RIGHT_X + 4,
CUSTOMER_BOX_Y + 9);
    }

    // ORDER box, right next to the customer.
    fill_rectangle(rgb(255, 255, 255), ORDER_BOX_X, CUSTOMER_BOX_Y,
        ORDER_BOX_WIDTH, CUSTOMER_BOX_HEIGHT);
    draw_rectangle(rgb(231, 207, 207), ORDER_BOX_X, CUSTOMER_BOX_Y,
        ORDER_BOX_WIDTH, CUSTOMER_BOX_HEIGHT);

    if (leaving)
    {
        draw_game_text("Next customer...", rgb(126, 116, 116), text_x, CUSTOMER_BOX_Y + 60);
        return;
    }

    if (game.order_just_completed)
    {
        draw_game_text("ORDER COMPLETE!", rgb(105, 157, 126), text_x, CUSTOMER_BOX_Y + 42);
        draw_game_text("Thank you!", rgb(91, 78, 82), text_x, CUSTOMER_BOX_Y + 72);
        return;
    }

    draw_game_text("CUSTOMER ORDER", rgb(121, 69, 84), text_x, CUSTOMER_BOX_Y + 12);

    string order_text = number_text(current_order_quantity(game));
    order_text += " x level ";
    order_text += number_text(current_order_tier(game) + 1);
    order_text += " ";
    order_text += item_name(current_order_category(game));

    draw_game_text(order_text, rgb(91, 78, 82), text_x, CUSTOMER_BOX_Y + 42);
```

```
2064
2065        int first_ready = game.delivered_count;
2066        if (game.mode == RECIPE_MODE)
2067        {
2068            first_ready = game.recipe_first_delivered;
2069        }
2070
2071        string ready_text = "Ready: ";
2072        ready_text += number_text(first_ready);
2073        ready_text += " / ";
2074        ready_text += number_text(current_order_quantity(game));
2075
2076        draw_game_text(ready_text, rgb(126, 116, 116), text_x, CUSTOMER_BOX_Y + 68);
2077
2078        if (game.mode == RECIPE_MODE)
2079        {
2080            string second_order_text = "+ ";
2081            second_order_text += number_text(current_order_secondary_quantity(game));
2082            second_order_text += " x level ";
2083            second_order_text += number_text(current_order_secondary_tier(game) + 1);
2084            second_order_text += " ";
2085            second_order_text += item_name(current_order_secondary_category(game));
2086
2087            draw_game_text(second_order_text, rgb(91, 78, 82), text_x, CUSTOMER_BOX_Y + 96);
2088
2089            string second_ready_text = "Ready: ";
2090            second_ready_text += number_text(game.recipe_second_delivered);
2091            second_ready_text += " / ";
2092            second_ready_text += number_text(current_order_secondary_quantity(game));
2093
2094            draw_game_text(second_ready_text, rgb(126, 116, 116), text_x, CUSTOMER_BOX_Y + 120);
2095        }
2096    }
2097
2098    // Draw the progress bar used for the Rush Hour timer and the Recipe moves.
2099    static void draw_bar(color bar_colour, int x, int y, double fraction)
2100    {
2101        if (fraction < 0) fraction = 0;
2102        if (fraction > 1) fraction = 1;
2103
2104        fill_rectangle(rgb(245, 229, 225), x, y, 260, 19);
2105        fill_rectangle(bar_colour, x, y, (int)(260.0 * fraction), 19);
2106    }
2107
2108    // The panel under the order. It shows what matters in the chosen mode:
2109    //   Rush Hour - the customer's hearts and the 60 second timer
2110    //   Recipe - how many moves are left
2111    //   Beginner/Classic - customer number and no timer.
2112    static void draw_status_panel(const game_state &game, unsigned int now)
2113    {
```

```
2114      int text_x = RIGHT_X + 20;
2115      fill_rectangle(rgb(255, 255, 255), RIGHT_X, STATUS_BOX_Y, RIGHT_WIDTH,
          STATUS_BOX_HEIGHT);
2116      draw_rectangle(rgb(236, 218, 218), RIGHT_X, STATUS_BOX_Y, RIGHT_WIDTH,
          STATUS_BOX_HEIGHT);
2117
2118      if (game.mode == RUSH_MODE)
2119      {
2120          draw_game_text("CUSTOMER SATISFACTION", rgb(121, 69, 84), text_x, STATUS_BOX_Y +
          15);
2121
2122          for (int heart = 0; heart < MAX_HEARTS; heart++)
2123          {
2124              color heart_colour = rgb(221, 211, 211);
2125              if (heart < game.satisfaction)
2126                  heart_colour = rgb(229, 87, 116);
2127              fill_circle(heart_colour, RIGHT_X + 39 + heart * 36, STATUS_BOX_Y + 59, 12);
2128          }
2129
2130          unsigned int left = current_order_time_left(game, now);
2131          color timer_colour = rgb(228, 167, 101);
2132          if (left < 15000)
2133              timer_colour = rgb(221, 94, 103);
2134
2135          draw_bar(timer_colour, text_x, STATUS_BOX_Y + 86, (double)left / ORDER_TIME_LIMIT);
2136          draw_game_text(number_text(left / 1000) + " seconds left", rgb(91, 78, 82), text_x,
          STATUS_BOX_Y + 112);
2137      }
2138      else if (game.mode == RECIPE_MODE)
2139      {
2140          draw_game_text("MOVES LEFT", rgb(121, 69, 84), text_x, STATUS_BOX_Y + 15);
2141          draw_game_text(number_text(game.moves_left) + " of " +
          number_text(current_move_limit(game)) + " moves", rgb(91, 78, 82), text_x, STATUS_BOX_Y +
          48);
2142
2143          color moves_colour = rgb(165, 117, 179);
2144          if (game.moves_left <= 5)
2145              moves_colour = rgb(221, 94, 103);
2146
2147          draw_bar(moves_colour, text_x, STATUS_BOX_Y + 78, (double)game.moves_left /
          current_move_limit(game));
2148          draw_game_text("Out of moves ends the game.", rgb(126, 116, 116), text_x,
          STATUS_BOX_Y + 106);
2149      }
2150      else
2151      {
2152          draw_game_text("CUSTOMER " + number_text(game.order_index + 1) + " OF " +
          number_text(order_count_for_mode(game.mode)), rgb(121, 69, 84), text_x, STATUS_BOX_Y + 15);
2153          draw_game_text("No timer in this mode.", rgb(91, 78, 82), text_x, STATUS_BOX_Y +
          55);
2154          draw_game_text("Take your time!", rgb(126, 116, 116), text_x, STATUS_BOX_Y + 85);
```

```
2155     }
2156 }
2157
2158 // Show the selected food and its requested level in every game mode.
2159 static void draw_selection_panel(const game_state &game)
2160 {
2161     fill_rectangle(rgb(255, 255, 255), RIGHT_X, 470, RIGHT_WIDTH, 130);
2162     draw_rectangle(rgb(236, 218, 218), RIGHT_X, 470, RIGHT_WIDTH, 130);
2163     draw_game_text("SELECTED FOOD", rgb(121, 69, 84), RIGHT_X + 20, 482);
2164
2165     if (game.selected_row >= 0 && game.selected_column >= 0)
2166     {
2167         board_item selected = game.board[game.selected_row][game.selected_column];
2168         draw_item(selected.category, selected.tier, RIGHT_X + 8, 488);
2169         draw_game_text(item_name(selected.category), rgb(91, 78, 82), RIGHT_X + 126, 502);
2170         draw_game_text("Level " + number_text(selected.tier + 1), rgb(91, 78, 82), RIGHT_X +
    126, 528);
2171         int needed_tier = -1;
2172         if (selected.category == current_order_category(game))
2173             needed_tier = current_order_tier(game);
2174         else if (game.mode == RECIPE_MODE && selected.category == current_order_second-
    ary_category(game))
2175             needed_tier = current_order_secondary_tier(game);
2176
2177         if (needed_tier >= 0)
2178             draw_game_text("Order needs level " + number_text(needed_tier + 1) + ".",
    rgb(126, 116, 116), RIGHT_X + 126, 554);
2179         else
2180             draw_game_text("Not part of this order.", rgb(126, 116, 116), RIGHT_X + 126,
    554);
2181     }
2182     else if (game.last_action != "")
2183     {
2184         draw_game_text(game.last_action, rgb(91, 78, 82), RIGHT_X + 20, 528);
2185     }
2186     else
2187     {
2188         draw_game_text("Nothing selected.", rgb(91, 78, 82), RIGHT_X + 20, 522);
2189         draw_game_text("Click a food tile on the tray to inspect it.", rgb(126, 116, 116),
    RIGHT_X + 20, 550);
2190     }
2191 }
2192
2193 // Draw the game board, customer order, and status information.
2194 static void draw_play(const game_state &game, unsigned int now)
2195 {
2196     draw_header(mode_name(game.mode));
2197
2198     // Keep a visible way back to the mode menu during play.
2199     fill_rectangle(rgb(121, 69, 84), BACK_X, BACK_Y, BACK_WIDTH, BACK_HEIGHT);
2200     draw_game_text("BACK", rgb(255, 255, 255), BACK_X + 33, BACK_Y + 10);
```

```
2201
2202        draw_game_text("SCORE  " + number_text(game.score), rgb(121, 69, 84), 690, 25);
2203        draw_game_text("MOVES  " + number_text(game.moves), rgb(121, 69, 84), 690, 66);
2204
2205        fill_rectangle(rgb(121, 69, 84), PAUSE_BUTTON_X, PAUSE_BUTTON_Y, PAUSE_BUTTON_WIDTH,
    PAUSE_BUTTON_HEIGHT);
2206        string pause_label = game.paused ? "RESUME" : "PAUSE";
2207        draw_game_text(pause_label, rgb(255, 255, 255), centered_ui_text_x(pause_label,
    PAUSE_BUTTON_X, PAUSE_BUTTON_WIDTH), PAUSE_BUTTON_Y + 10);
2208
2209        draw_game_text("BAKERY TRAY", rgb(121, 69, 84), BOARD_X, 130);
2210
2211        for (int row = 0; row < GRID_SIZE; row++)
2212        {
2213            for (int col = 0; col < GRID_SIZE; col++)
2214            {
2215                int x = BOARD_X + col * (CELL_SIZE + CELL_GAP);
2216                int y = BOARD_Y + row * (CELL_SIZE + CELL_GAP);
2217                fill_rectangle(rgb(255, 255, 255), x, y, CELL_SIZE, CELL_SIZE);
2218                draw_rectangle(rgb(239, 224, 220), x, y, CELL_SIZE, CELL_SIZE);
2219
2220                board_item item = game.board[row][col];
2221                draw_item(item.category, item.tier, x, y);
2222                if (row == game.selected_row && col == game.selected_column)
2223                {
2224                    fill_rectangle(rgba_color(211, 93, 127, 45), x, y, CELL_SIZE, CELL_SIZE);
2225                    draw_rectangle(rgb(211, 93, 127), x + 2, y + 2, CELL_SIZE - 4, CELL_SIZE -
    4);
2226                    draw_rectangle(rgb(211, 93, 127), x + 4, y + 4, CELL_SIZE - 8, CELL_SIZE -
    8);
2227                }
2228            }
2229        }
2230
2231        draw_generators();
2232        draw_beginner_guidance(game);
2233        draw_customer_and_order(game, now);
2234        draw_status_panel(game, now);
2235
2236        draw_selection_panel(game);
2237
2238        if (game.paused)
2239        {
2240            fill_rectangle(rgba_color(54, 42, 47, 155), 0, 112, WINDOW_WIDTH, WINDOW_HEIGHT -
    112);
2241            fill_rectangle(rgb(255, 247, 241), PAUSE_PANEL_X, PAUSE_PANEL_Y, PAUSE_PANEL_WIDTH,
    PAUSE_PANEL_HEIGHT);
2242            draw_rectangle(rgb(211, 93, 127), PAUSE_PANEL_X, PAUSE_PANEL_Y, PAUSE_PANEL_WIDTH,
    PAUSE_PANEL_HEIGHT);
2243            draw_game_text("PAUSED", rgb(121, 69, 84), centered_ui_text_x("PAUSED",
    PAUSE_PANEL_X, PAUSE_PANEL_WIDTH), PAUSE_PANEL_Y + 32);
```

```
2244        draw_game_text("Click RESUME or press P to continue.", rgb(91, 78, 82),
    centered_ui_text_x("Click RESUME or press P to continue.", PAUSE_PANEL_X,
    PAUSE_PANEL_WIDTH), PAUSE_PANEL_Y + 78);
2245    }
2246 }
2247
2248 // Draw one centred line of text inside the end-screen panel.
2249 static void draw_end_line(const string &text, color colour, int y)
2250 {
2251     draw_game_text(text, colour, centered_ui_text_x(text, END_BOX_X, END_BOX_WIDTH), y);
2252 }
2253
2254 // Draw the win or game-over screen on a white panel so the text is easy to read.
2255 static void draw_end(const game_state &game)
2256 {
2257     draw_header(mode_name(game.mode) + " RESULTS");
2258     fill_rectangle(rgb(255, 255, 255), END_BOX_X, END_BOX_Y, END_BOX_WIDTH, END_BOX_HEIGHT);
2259     draw_rectangle(rgb(231, 207, 207), END_BOX_X, END_BOX_Y, END_BOX_WIDTH, END_BOX_HEIGHT);
2260
2261     if (game.screen == WON_SCREEN)
2262     {
2263         draw_end_line("FESTIVAL SAVED!", rgb(105, 157, 126), END_BOX_Y + 36);
2264         draw_end_line("You served every order at Mia's bakery.", rgb(91, 78, 82), END_BOX_Y
    + 86);
2265     }
2266     else if (game.lost_by_timer)
2267     {
2268         draw_end_line("THE CUSTOMER LEFT", rgb(164, 78, 89), END_BOX_Y + 36);
2269         draw_end_line("They waited too long for their order.", rgb(91, 78, 82), END_BOX_Y +
    86);
2270     }
2271     else if (game.lost_by_move_limit)
2272     {
2273         draw_end_line("OUT OF MOVES", rgb(164, 78, 89), END_BOX_Y + 36);
2274         draw_end_line("You ran out of moves for this recipe.", rgb(91, 78, 82), END_BOX_Y +
    86);
2275     }
2276     else
2277     {
2278         draw_end_line("NO MOVES LEFT", rgb(164, 78, 89), END_BOX_Y + 36);
2279         draw_end_line("The board is full and nothing can merge.", rgb(91, 78, 82), END_BOX_Y
    + 86);
2280     }
2281
2282
2283     draw_end_line("Final score: " + number_text(game.score), rgb(91, 78, 82), END_BOX_Y +
    126);
2284     if (game.mode != BEGINNER_MODE)
2285     {
2286         if (game.new_high_score)
```

```
            draw_end_line("NEW PERSONAL HIGHEST MARK!", rgb(105, 157, 126), END_BOX_Y + 157);
        else
            draw_end_line("Personal highest mark: " + number_text(high_score_for_mode(game.mode)), rgb(146, 109, 116), END_BOX_Y + 157);
    }

    fill_rectangle(rgb(121, 69, 84), END_REPLAY_BUTTON_X, END_BUTTON_Y, END_BUTTON_WIDTH, END_BUTTON_HEIGHT);
    draw_rectangle(rgb(121, 69, 84), END_REPLAY_BUTTON_X, END_BUTTON_Y, END_BUTTON_WIDTH, END_BUTTON_HEIGHT);
    draw_game_text("PLAY AGAIN", rgb(255, 255, 255), centered_ui_text_x("PLAY AGAIN", END_REPLAY_BUTTON_X, END_BUTTON_WIDTH), END_BUTTON_Y + 13);

    fill_rectangle(rgb(214, 171, 157), END_MENU_BUTTON_X, END_BUTTON_Y, END_BUTTON_WIDTH, END_BUTTON_HEIGHT);
    draw_rectangle(rgb(214, 171, 157), END_MENU_BUTTON_X, END_BUTTON_Y, END_BUTTON_WIDTH, END_BUTTON_HEIGHT);
    draw_game_text("BACK TO MENU", rgb(255, 255, 255), centered_ui_text_x("BACK TO MENU", END_MENU_BUTTON_X, END_BUTTON_WIDTH), END_BUTTON_Y + 13);
}

// Draw the background and whichever screen is currently active.
void draw_game(const game_state &game, unsigned int now)
{
    clear_screen(rgb(255, 247, 241));
    // The background picture is already 960 x 700, the same size as the window.
    draw_bitmap("bakery_background", 0, 0);
    if (game.screen == MODE_MENU)
    {
        draw_menu(game);
    }
    else if (game.screen == STORY_SCREEN)
    {
        draw_story(game);
    }
    else if (game.screen == PLAYING_SCREEN)
    {
        draw_play(game, now);
    }
    else
    {
        draw_end(game);
    }
}

// ========= File: src/main.cpp ==========
// src/main.cpp
#include "splashkit.h"
#include "game_state.h"
#include "game_logic.h"
```

```
#include "game_view.h"
#include "input.h"
#include "sound_manager.h"
#include "score_manager.h"
#include "game_layout.h"

static void record_round_if_finished(game_state &game)
{
    bool has_ended = (game.screen == WON_SCREEN || game.screen == GAME_OVER_SCREEN);
    if (has_ended && !game.result_recorded)
    {
        game.new_high_score = record_high_score(game.mode, game.score);
        game.result_recorded = true;
    }
}

int main()
{
    game_state game;
    start_new_game(game);
    open_window("Sweet Stack - Cake Merge", WINDOW_WIDTH, WINDOW_HEIGHT);
    load_game_art();
    load_game_sounds();
    play_story_voice(game, game.story_line);
    load_background_music();
    play_background_music();

    while (!quit_requested())
    {
        // Handle input, update game rules, then draw the current screen.
        process_events();

        unsigned int now = current_ticks();
        update_game(game, now);
        record_round_if_finished(game);
        handle_key_presses(game, now);
        if (mouse_clicked(LEFT_BUTTON))
        {
            handle_mouse_click(game, mouse_x(), mouse_y(), now);
        }
        record_round_if_finished(game);
        draw_game(game, now);
        refresh_screen(60);
    }

    close_all_windows();
    return 0;
}

// ========= File: tests/test_sweet_stack.cpp ==========
```

```
// tests/test_sweet_stack.cpp
#include "splashkit.h"
#include "game_state.h"
#include "game_logic.h"
#include "sound_manager.h"
#include "score_manager.h"
#include "game_layout.h"
#include "utilities.h"
#include "input.h"

// Print PASS or FAIL for one check.
void check(bool condition, string test_name)
{
    if (condition)
    {
        write_line("PASS: " + test_name);
    }
    else
    {
        write_line("FAIL: " + test_name);
    }
}

// Count how many board squares hold food.
int count_items(const game_state &game)
{
    int count = 0;
    for (int row = 0; row < GRID_SIZE; row++)
        for (int col = 0; col < GRID_SIZE; col++)
            if (game.board[row][col].category != NO_ITEM)
                count++;
    return count;
}

// Put one item on the board by hand.
void place(game_state &game, int row, int col, item_category category, int tier)
{
    game.board[row][col].category = category;
    game.board[row][col].tier = tier;
}

void test_screen_flow()
{
    game_state game = game_state();   // every value starts at zero
    start_new_game(game);
    check(game.screen == STORY_SCREEN, "program starts on the story screen");

    advance_screen(game, 0);
    advance_screen(game, 0);
```

```
2429      check(game.screen == STORY_SCREEN, "story is still showing before the last line is
finished");
2430      advance_screen(game, 0);
2431      check(game.screen == MODE_MENU, "finishing the story shows the mode menu");
2432
2433      choose_mode(game, RUSH_MODE, 1000);
2434      check(game.screen == PLAYING_SCREEN, "choosing a mode starts gameplay immediately");
2435      check(game.mode == RUSH_MODE, "chosen mode is stored");
2436      check(game.order_started_at == 1000, "timer starts when the mode starts");
2437
2438      show_mode_menu(game);
2439      check(game.screen == MODE_MENU, "Back returns to the mode menu (not the story)");
2440
2441      choose_mode(game, RECIPE_MODE, 2000);
2442      check(game.screen == PLAYING_SCREEN && game.moves_left == 12, "choosing another mode
starts it immediately with its own settings");
2443  }
2444
2445  void test_generators()
2446  {
2447      game_state game = game_state();   // every value starts at zero
2448      choose_mode(game, BEGINNER_MODE, 0);
2449
2450      check(generate_item(game, DOUGH), "dough generator succeeds");
2451      check(count_items(game) == 1, "dough generator adds exactly one item");
2452
2453      check(generate_item(game, TART), "tart generator succeeds");
2454      check(generate_item(game, CAKE), "cake generator succeeds");
2455      check(count_items(game) == 3, "three generator clicks give three items");
2456
2457      bool all_tier_zero = true;
2458      for (int row = 0; row < GRID_SIZE; row++)
2459          for (int col = 0; col < GRID_SIZE; col++)
2460              if (game.board[row][col].category != NO_ITEM && game.board[row][col].tier != 0)
2461                  all_tier_zero = false;
2462      check(all_tier_zero, "new items start at tier 0");
2463
2464      for (int i = 0; i < 13; i++)
2465          generate_item(game, DOUGH);
2466      check(count_items(game) == GRID_SIZE * GRID_SIZE, "board can be filled completely");
2467      check(!generate_item(game, DOUGH), "generator fails when the board is full");
2468  }
2469
2470  void test_move_and_merge()
2471  {
2472      game_state game = game_state();   // every value starts at zero
2473      choose_mode(game, CLASSIC_MODE, 0);
2474
2475      place(game, 0, 0, DOUGH, 0);
2476      select_or_move_cake(game, 0, 0, 0);
```

```
    select_or_move_cake(game, 2, 2, 0);
    check(game.board[2][2].category == DOUGH && game.board[0][0].category == NO_ITEM, "item moves into an empty square");
    check(game.moves == 1, "a move is counted");

    place(game, 0, 0, DOUGH, 0);
    select_or_move_cake(game, 0, 0, 0);
    select_or_move_cake(game, 2, 2, 0);
    check(game.board[2][2].tier == 1 && game.board[0][0].category == NO_ITEM, "two matching items merge into a higher tier");
    check(game.score == 10, "merge adds score");

    place(game, 0, 0, TART, 1);
    select_or_move_cake(game, 0, 0, 0);
    select_or_move_cake(game, 2, 2, 0);
    check(game.board[2][2].category == DOUGH && game.board[0][0].category == TART, "different foods do not merge");
}

// Remove every food from the board so a test starts from a known position.
void empty_board(game_state &game)
{
    for (int row = 0; row < 4; row++)
    {
        for (int column = 0; column < 4; column++)
        {
            game.board[row][column].category = NO_ITEM;
            game.board[row][column].tier = 0;
        }
    }
}

// Finish the current Beginner/Classic/Rush order by merging two foods
// that are one tier lower than the tier the customer asked for.
void finish_current_order(game_state &game, unsigned int now)
{
    item_category food = current_order_category(game);
    int tier = current_order_tier(game);

    empty_board(game);
    game.delivered_count = current_order_quantity(game) - 1;
    place(game, 0, 0, food, tier - 1);
    place(game, 0, 1, food, tier - 1);
    select_or_move_cake(game, 0, 0, now);
    select_or_move_cake(game, 0, 1, now);
}

// Move one food from one square to another empty square.
void move_food(game_state &game, int from_row, int from_column, int to_row, int to_column)
{
    select_or_move_cake(game, from_row, from_column, 0);
```

```
2525      select_or_move_cake(game, to_row, to_column, 0);
2526  }
2527
2528  void test_order_complete()
2529  {
2530      game_state game = game_state();   // every value starts at zero
2531      choose_mode(game, BEGINNER_MODE, 0);
2532
2533      // First order needs one tier 2 dough. Merge two tier 1 dough to make it.
2534      place(game, 0, 0, DOUGH, 1);
2535      place(game, 0, 1, DOUGH, 1);
2536      select_or_move_cake(game, 0, 0, 0);
2537      select_or_move_cake(game, 0, 1, 0);
2538      check(game.order_just_completed, "making the ordered item completes the order");
2539      check(game.score == 120, "completed order adds the order bonus plus merge score");
2540  }
2541
2542  void test_recipe_rejects_higher_tier()
2543  {
2544      game_state game = game_state();
2545      choose_mode(game, RECIPE_MODE, 0);
2546
2547      game.orders[0].category = DOUGH;
2548      game.orders[0].tier = 1;
2549      game.orders[0].quantity = 1;
2550      game.orders[0].secondary_category = CAKE;
2551      game.orders[0].secondary_tier = 1;
2552      game.orders[0].secondary_quantity = 1;
2553      game.orders[0].move_limit = 24;
2554      game.moves_left = 24;
2555
2556      empty_board(game);
2557      place(game, 0, 0, DOUGH, 1);
2558      place(game, 0, 1, DOUGH, 1);
2559      select_or_move_cake(game, 0, 0, 0);
2560      select_or_move_cake(game, 0, 1, 0);
2561      check(game.recipe_first_delivered == 0 && !game.order_just_completed,
2562            "a higher-tier merge does not satisfy a lower-tier recipe item");
2563
2564      empty_board(game);
2565      place(game, 0, 0, DOUGH, 0);
2566      place(game, 0, 1, DOUGH, 0);
2567      select_or_move_cake(game, 0, 0, 0);
2568      select_or_move_cake(game, 0, 1, 0);
2569      check(game.recipe_first_delivered == 1 && !game.order_just_completed,
2570            "an exact-tier recipe merge advances only its matching item");
2571  }
2572
2573  void test_recipe_food_collection_delay()
2574  {
```

```
2575      game_state game = game_state();
2576      choose_mode(game, RECIPE_MODE, 0);
2577
2578      game.orders[0].category = DOUGH;
2579      game.orders[0].tier = 1;
2580      game.orders[0].quantity = 1;
2581      game.orders[0].secondary_category = CAKE;
2582      game.orders[0].secondary_tier = 1;
2583      game.orders[0].secondary_quantity = 1;
2584
2585      empty_board(game);
2586      place(game, 0, 0, DOUGH, 0);
2587      place(game, 0, 1, DOUGH, 0);
2588      select_or_move_cake(game, 0, 0, 2000);
2589      select_or_move_cake(game, 0, 1, 2000);
2590
2591      check(game.recipe_first_delivered == 1 && !game.order_just_completed,
2592            "Recipe Challenge counts one matching requirement independently");
2593      check(game.board[0][1].category == DOUGH && game.recipe_food_pending[0][1],
2594            "delivered recipe food remains visible during collection delay");
2595
2596      int moves_after_delivery = game.moves;
2597      select_or_move_cake(game, 0, 1, 2500);
2598      check(game.board[0][1].category == DOUGH && game.moves == moves_after_delivery,
2599            "food being collected cannot be selected or merged again");
2600
2601      update_game(game, 2000 + ORDER_COMPLETE_MESSAGE_TIME - 1);
2602      check(game.board[0][1].category == DOUGH,
2603            "recipe food remains until the normal one-second collection time");
2604      update_game(game, 2000 + ORDER_COMPLETE_MESSAGE_TIME);
2605      check(game.board[0][1].category == NO_ITEM && !game.recipe_food_pending[0][1],
2606            "recipe food is collected after the normal one-second delay");
2607  }
2608
2609  void test_beginner_has_no_high_mark()
2610  {
2611      check(high_score_for_mode(BEGINNER_MODE) == 0,
2612            "Beginner does not display a personal highest mark");
2613      check(!record_high_score(BEGINNER_MODE, 999999),
2614            "Beginner scores are not recorded as personal highest marks");
2615  }
2616  void test_customer_leaving()
2617  {
2618      game_state game = game_state();   // every value starts at zero
2619      choose_mode(game, BEGINNER_MODE, 0);
2620
2621      finish_current_order(game, 1000);
2622      check(game.order_just_completed && game.screen == PLAYING_SCREEN, "finished order starts
      the customer transition");
```

```
2623     check(game.order_index == 0, "order number stays the same until the next customer
arrives");
2624     check(!customer_has_left(game, 1500), "customer is still shown during the Order complete
message");
2625     check(customer_has_left(game, 1000 + ORDER_COMPLETE_MESSAGE_TIME), "customer has left
after the message");
2626 
2627     // The board is locked, so no second order can be completed by accident.
2628     int moves_before = game.moves;
2629     empty_board(game);
2630     place(game, 2, 0, DOUGH, 0);
2631     place(game, 2, 1, DOUGH, 0);
2632     select_or_move_cake(game, 2, 0, 1200);
2633     select_or_move_cake(game, 2, 1, 1200);
2634     check(game.board[2][0].category == DOUGH && game.board[2][1].category == DOUGH,"board is
locked while the customer is leaving");
2635     check(game.moves == moves_before, "locked clicks do not count as moves");
2636 
2637     update_game(game, 1000 + CUSTOMER_TRANSITION_TIME - 1);
2638     check(game.order_just_completed && game.order_index == 0, "next customer does not arrive
too early");
2639 
2640     update_game(game, 1000 + CUSTOMER_TRANSITION_TIME);
2641     check(!game.order_just_completed && game.order_index == 1, "next customer arrives with
the next order");
2642     check(game.delivered_count == 0, "new order starts with no progress");
2643 
2644     select_or_move_cake(game, 2, 0, 5000);
2645     select_or_move_cake(game, 2, 1, 5000);
2646     check(game.board[2][1].tier == 1, "board works again for the next customer");
2647 }
2648 
2649 void test_prebuilt_items_count_for_all_modes()
2650 {
2651     game_mode modes[4] = {BEGINNER_MODE, CLASSIC_MODE, RUSH_MODE, RECIPE_MODE};
2652     for (int mode_index = 0; mode_index < 4; mode_index++)
2653     {
2654         game_state game = game_state();
2655         choose_mode(game, modes[mode_index], 1000);
2656 
2657         game.order_index = 0;
2658         game.order_just_completed = true;
2659         game.order_completed_at = 1000;
2660         game.served_food_removed = true;
2661         game.orders[1].category = TART;
2662         game.orders[1].tier = 2;
2663         game.orders[1].quantity = 1;
2664 
2665         place(game, 0, 0, TART, 1); // Wrong level must not count.
2666         place(game, 0, 1, TART, 2); // Prepared exact match for the next order.
2667 
```

```
        if (modes[mode_index] == RECIPE_MODE)
        {
            game.orders[1].secondary_category = CAKE;
            game.orders[1].secondary_tier = 1;
            game.orders[1].secondary_quantity = 1;
            game.orders[1].move_limit = 16;
            place(game, 1, 0, CAKE, 0); // Wrong level must not count.
            place(game, 1, 1, CAKE, 1); // Prepared exact match for the second requirement.
        }

        update_game(game, 1000 + CUSTOMER_TRANSITION_TIME);

        if (modes[mode_index] == RECIPE_MODE)
        {
            check(game.order_index == 1 && game.recipe_first_delivered == 1 &&
                      game.recipe_second_delivered == 1 && game.order_just_completed,
                  "Recipe Challenge counts prepared exact matches when the next customer arrives");
            check(game.recipe_food_pending[0][1] && game.recipe_food_pending[1][1] &&
                      game.board[0][0].category == TART && game.board[1][0].category == CAKE,
                  "Recipe Challenge delays collection and ignores prepared items at the wrong level");
        }
        else
        {
            check(game.order_index == 1 && game.delivered_count == 1 && game.order_just_completed,
                  "Beginner, Classic Bakery and Rush Hour count a prepared exact match on arrival");
            check(game.board[0][1].category == TART && game.board[0][0].category == TART,
                  "regular modes keep prepared food visible during customer acknowledgement");
        }
    }
}
void test_final_order_wins()
{
    game_state game = game_state();   // every value starts at zero
    choose_mode(game, CLASSIC_MODE, 0);
    unsigned int now = 0;

    for (int i = 0; i < order_count_for_mode(game.mode) - 1; i++)
    {
        finish_current_order(game, now);
        update_game(game, now + CUSTOMER_TRANSITION_TIME);
        now += 10000;
    }
    check(game.order_index == order_count_for_mode(game.mode) - 1 && game.screen == PLAYING_SCREEN, "first two orders each bring a new customer");

```

```
2712      finish_current_order(game, now);
2713      check(game.screen == WON_SCREEN, "final order shows the win screen");
2714      check(!game.order_just_completed, "final order has no customer transition");
2715  }
2716
2717  void test_rush_hour_timer()
2718  {
2719      check(ORDER_TIME_LIMIT == 30000, "Rush Hour gives each customer 30 seconds");
2720      check(CUSTOMER_HEART_TIME == 10000, "a heart fades every 10 seconds");
2721      check(MAX_HEARTS * CUSTOMER_HEART_TIME == ORDER_TIME_LIMIT, "three hearts last exactly
      one order");
2722
2723      game_state game = game_state();
2724      choose_mode(game, RUSH_MODE, 0);
2725      check(game.satisfaction == 3, "Rush Hour customer starts with three hearts");
2726      check(current_order_time_left(game, 5000) == 25000, "timer counts down from 30
      seconds");
2727
2728      // One heart is lost every 10 seconds (the last heart goes when time is up).
2729      update_game(game, 9999);
2730      check(game.satisfaction == 3, "still three hearts just before 10 seconds");
2731      update_game(game, 10000);
2732      check(game.satisfaction == 2, "first heart is lost at 10 seconds");
2733      update_game(game, 19999);
2734      check(game.satisfaction == 2, "still two hearts just before 20 seconds");
2735      update_game(game, 20000);
2736      check(game.satisfaction == 1, "second heart is lost at 20 seconds");
2737      update_game(game, 29999);
2738      check(game.screen == PLAYING_SCREEN && game.satisfaction == 1, "customer waits for the
      full 30 seconds");
2739      update_game(game, ORDER_TIME_LIMIT);
2740      check(game.screen == GAME_OVER_SCREEN && game.lost_by_timer, "customer leaves at 30
      seconds and the game is over");
2741
2742      // Hearts and timer are frozen while the customer is leaving.
2743      choose_mode(game, RUSH_MODE, 0);
2744      finish_current_order(game, 15000);
2745      check(game.satisfaction == 2, "customer keeps the hearts they had when served");
2746      update_game(game, 15500);
2747      check(game.satisfaction == 2, "hearts do not change while the customer is leaving");
2748      check(current_order_time_left(game, 95000) == ORDER_TIME_LIMIT - 15000, "timer is frozen
      during the transition");
2749
2750      // A late order must not be punished by the transition time.
2751      choose_mode(game, RUSH_MODE, 0);
2752      finish_current_order(game, 29000);
2753      update_game(game, 30500);
2754      check(game.screen == PLAYING_SCREEN, "no game over while the customer is leaving");
2755
2756      // The next customer gets full hearts and a full timer.
2757      update_game(game, 29000 + CUSTOMER_TRANSITION_TIME);
```

```
2758      check(game.satisfaction == 3, "next customer starts with three hearts");
2759      check(current_order_time_left(game, 29000 + CUSTOMER_TRANSITION_TIME) ==
ORDER_TIME_LIMIT, "next customer gets a full timer");
2760  }
2761
2762  void test_recipe_move_limit()
2763  {
2764      game_state game = game_state();   // every value starts at zero
2765      choose_mode(game, RECIPE_MODE, 0);
2766      int limit = game.moves_left;
2767      check(limit == 12, "Recipe Challenge starts with 12 moves for the first customer");
2768
2769      empty_board(game);
2770      place(game, 0, 0, DOUGH, 0);
2771      for (int i = 0; i < limit - 1; i++)
2772      {
2773          if (i % 2 == 0)
2774              move_food(game, 0, 0, 0, 1);
2775          else
2776              move_food(game, 0, 1, 0, 0);
2777      }
2778      check(game.moves_left == 1 && game.screen == PLAYING_SCREEN, "moves are counted down
while the game continues");
2779
2780      if ((limit - 1) % 2 == 0)
2781          move_food(game, 0, 0, 0, 1);
2782      else
2783          move_food(game, 0, 1, 0, 0);
2784      check(game.screen == GAME_OVER_SCREEN && game.lost_by_move_limit, "using the last move
without finishing the recipe ends the game");
2785
2786      // Finishing the order on the very last move must not end the game.
2787      choose_mode(game, RECIPE_MODE, 0);
2788      empty_board(game);
2789      game.moves_left = 1;
2790      game.recipe_first_delivered = current_order_quantity(game) - 1;
2791      game.recipe_second_delivered = current_order_secondary_quantity(game);
2792      place(game, 0, 0, current_order_category(game), current_order_tier(game) - 1);
2793      place(game, 0, 1, current_order_category(game), current_order_tier(game) - 1);
2794      select_or_move_cake(game, 0, 0, 0);
2795      select_or_move_cake(game, 0, 1, 0);
2796      check(game.order_just_completed && game.screen == PLAYING_SCREEN, "finishing an order on
the last move is not a loss");
2797      update_game(game, ORDER_COMPLETE_MESSAGE_TIME);
2798      check(game.board[0][1].category == NO_ITEM, "recipe food disappears when the customer
leaves");
2799  }
2800
2801  void test_served_food_disappears()
2802  {
2803      game_state game = game_state();   // every value starts at zero
```

```
2804      choose_mode(game, BEGINNER_MODE, 0);
2805      finish_current_order(game, 1000);
2806      place(game, 3, 3, TART, 0);   // a food that is not part of the order
2807
2808      item_category food = current_order_category(game);
2809      int tier = current_order_tier(game);
2810
2811      check(game.board[0][1].category == food && game.board[0][1].tier == tier, "ordered food
          is still on the board while the customer is thanked");
2812
2813      update_game(game, 1000 + ORDER_COMPLETE_MESSAGE_TIME - 1);
2814      check(game.board[0][1].category == food, "food stays until the customer leaves");
2815
2816      update_game(game, 1000 + ORDER_COMPLETE_MESSAGE_TIME);
2817      check(game.board[0][1].category == NO_ITEM, "ordered food disappears when the customer
          leaves");
2818      check(game.board[3][3].category == TART, "other foods stay on the board");
2819  }
2820
2821  // Beginner, Classic and Recipe have no hearts and no timer.
2822  void test_hearts_only_in_rush_hour()
2823  {
2824      game_state game = game_state();   // every value starts at zero
2825      choose_mode(game, BEGINNER_MODE, 0);
2826      update_game(game, 500000);
2827      check(game.screen == PLAYING_SCREEN, "Beginner has no timer");
2828
2829      choose_mode(game, CLASSIC_MODE, 0);
2830      update_game(game, 500000);
2831      check(game.screen == PLAYING_SCREEN, "Classic Bakery has no timer");
2832
2833      choose_mode(game, RECIPE_MODE, 0);
2834      update_game(game, 500000);
2835      check(game.screen == PLAYING_SCREEN, "Recipe Challenge has no timer");
2836      check(current_move_limit(game) == game.moves_left, "Recipe Challenge starts with the
          full move limit");
2837  }
2838
2839  void test_back_to_menu()
2840  {
2841      game_state game = game_state();   // every value starts at zero
2842      choose_mode(game, RUSH_MODE, 0);
2843      finish_current_order(game, 1000);
2844
2845      show_mode_menu(game);
2846      check(game.screen == MODE_MENU, "Back returns to the mode menu");
2847
2848      choose_mode(game, BEGINNER_MODE, 2000);
2849      check(game.screen == PLAYING_SCREEN && !game.order_just_completed && game.order_index ==
          0, "choosing a mode after Back starts a fresh game");
2850  }
```

```
2851
2852 void test_utilities()
2853 {
2854     check(number_text(0) == "0", "number_text of 0");
2855     check(number_text(1250) == "1250", "number_text of 1250");
2856
2857     check(is_in_range(5, 1, 10), "5 is inside 1 to 10");
2858     check(is_in_range(1, 1, 10) && is_in_range(10, 1, 10), "range includes both ends");
2859     check(!is_in_range(0, 1, 10) && !is_in_range(11, 1, 10), "values outside the range are
     rejected");
2860
2861     check(estimated_text_width("Hello") == 5 * TEXT_CHAR_WIDTH, "text width is estimated per
     character");
2862     check(centered_text_x("abcd", 100, 100) == 100 + (100 - 4 * TEXT_CHAR_WIDTH) / 2, "text
     is centred in a box");
2863
2864     check(point_in_rectangle(10, 10, 10, 10, 50, 20), "top-left corner is inside a
     rectangle");
2865     check(point_in_rectangle(59, 29, 10, 10, 50, 20), "bottom-right pixel is inside a
     rectangle");
2866     check(!point_in_rectangle(60, 20, 10, 10, 50, 20), "just right of a rectangle is
     outside");
2867     check(!point_in_rectangle(9, 20, 10, 10, 50, 20), "just left of a rectangle is
     outside");
2868 }
2869
2870 // The screen layout is shared by drawing and clicking, so check it makes sense.
2871 void test_layout()
2872 {
2873     check(BOARD_SIZE == 4 * CELL_SIZE + 3 * CELL_GAP, "board size matches cells and gaps");
2874     check(BOARD_Y + BOARD_SIZE <= GENERATOR_Y, "generator buttons start below the board");
2875     check(GENERATOR_Y + GENERATOR_HEIGHT <= WINDOW_HEIGHT, "generator buttons fit inside the
     window");
2876     check(DOUGH_BUTTON_X + GENERATOR_WIDTH <= TART_BUTTON_X, "dough and tart buttons do not
     overlap");
2877     check(TART_BUTTON_X + GENERATOR_WIDTH <= CAKE_BUTTON_X, "tart and cake buttons do not
     overlap");
2878     check(CAKE_BUTTON_X + GENERATOR_WIDTH <= BOARD_X + BOARD_SIZE, "buttons are no wider
     than the board");
2879     check(FOOD_IMAGE_SIZE + 2 * FOOD_MARGIN == CELL_SIZE, "food picture is centred in its
     square");
2880     check(GENERATOR_ICON_SIZE + 8 <= GENERATOR_HEIGHT, "generator icon fits inside its
     button");
2881     check(BOARD_X + BOARD_SIZE < RIGHT_X, "board does not overlap the right-hand panels");
2882     check(RIGHT_X + CUSTOMER_BOX_WIDTH <= ORDER_BOX_X, "customer box does not overlap the
     order box");
2883     check(ORDER_BOX_X + ORDER_BOX_WIDTH <= WINDOW_WIDTH, "order box fits inside the
     window");
2884     check(BACK_X + BACK_WIDTH <= WINDOW_WIDTH, "Back button fits inside the window");
2885
2886     // Text must keep a margin of at least 16 pixels inside its box.
```

```
2887     check(estimated_text_width("Orders ask for 1 or 2 items") + 2 * 20 <= MODE_CARD_WIDTH,
"longest mode-card line fits its card");
2888     check(estimated_text_width("+ 1 x level 4 tart") + 2 * 16 <= ORDER_BOX_WIDTH, "recipe
order line fits the order box");
2889     check(estimated_text_width("ORDER COMPLETE!") + 2 * 16 <= ORDER_BOX_WIDTH, "order-
complete message fits the order box");
2890     check(estimated_text_width("Click or press SPACE to choose a mode again.") + 2 * 16 <=
END_BOX_WIDTH, "end-screen hint fits its panel");
2891     check(MODE_RIGHT_X + MODE_CARD_WIDTH <= WINDOW_WIDTH, "mode cards fit inside the
window");
2892
2893     // Settings button and panel.
2894     check(SETTINGS_BUTTON_X + SETTINGS_BUTTON_WIDTH <= WINDOW_WIDTH, "Settings button fits
inside the window");
2895     check(SETTINGS_BOX_X + SETTINGS_BOX_WIDTH <= WINDOW_WIDTH && SETTINGS_BOX_Y +
SETTINGS_BOX_HEIGHT <= WINDOW_HEIGHT,
2896           "settings panel fits inside the window");
2897     check(SETTINGS_TOGGLE_X + SETTINGS_TOGGLE_WIDTH + 16 <= SETTINGS_BOX_X +
SETTINGS_BOX_WIDTH, "ON/OFF buttons fit inside the panel");
2898     check(SETTINGS_MUSIC_Y + SETTINGS_TOGGLE_HEIGHT <= SETTINGS_EFFECTS_Y, "the two switches
do not overlap");
2899     check(SETTINGS_EFFECTS_Y + SETTINGS_TOGGLE_HEIGHT < SETTINGS_CLOSE_Y, "Close button is
below the switches");
2900     check(SETTINGS_CLOSE_Y + SETTINGS_CLOSE_HEIGHT + 16 <= SETTINGS_BOX_Y +
SETTINGS_BOX_HEIGHT, "Close button fits inside the panel");
2901     check(SETTINGS_BOX_X + 30 + estimated_text_width("Background music") + 16 <=
SETTINGS_TOGGLE_X, "music label does not touch its button");
2902     check(estimated_text_width("Changes stay until you change them.") + 2 * 16 <=
SETTINGS_BOX_WIDTH, "settings hint fits the panel");
2903
2904     // Story screen text.
2905     check(estimated_text_width("Mia: Hi, I'm Mia, the baker who runs this shop.") + 2 * 30
<= 570, "story introduction fits the dialogue box");
2906     check(estimated_text_width("MIA - SHOP OWNER") + 2 * 16 <= STORY_NAME_WIDTH, "name plate
fits under the portrait");
2907     check(STORY_NAME_Y >= 416, "name plate does not cover the portrait");
2908 }
2909
2910 // True when one order has sensible values for the given mode.
2911 bool order_is_valid(const order_spec &order, game_mode mode)
2912 {
2913     bool ok = is_in_range(order.category, DOUGH, CAKE) &&
2914               is_in_range(order.tier, 1, MAX_ITEM_TIER) &&
2915               is_in_range(order.quantity, 1, 2) &&
2916               order.move_limit > 0;
2917
2918     if (mode == RECIPE_MODE)
2919     {
2920         ok = ok && is_in_range(order.secondary_category, DOUGH, CAKE) &&
2921              order.secondary_category != order.category &&
2922              is_in_range(order.secondary_tier, 1, MAX_ITEM_TIER) &&
2923              is_in_range(order.secondary_quantity, 1, 2);
```

```
    }
    else
    {
        ok = ok && order.secondary_category == NO_ITEM && order.secondary_quantity == 0;
    }

    return ok;
}

// Each of the three orders asks for a different food category.
bool orders_use_each_category_once(const game_state &game)
{
    int seen[4] = {0, 0, 0, 0};

    for (int i = 0; i < ORDER_COUNT; i++)
        seen[game.orders[i].category]++;

    return seen[DOUGH] == 1 && seen[TART] == 1 && seen[CAKE] == 1;
}

// Random orders are different every time, so these checks only test that
// every possible result is valid, never that two runs differ.
void test_random_orders()
{
    game_mode modes[3] = {CLASSIC_MODE, RUSH_MODE, RECIPE_MODE};
    string names[3] = {"Classic Bakery", "Rush Hour", "Recipe Challenge"};

    for (int m = 0; m < 3; m++)
    {
        bool all_valid = true;
        bool each_category_once = true;
        bool old_orders_replaced = true;
        bool moves_match_first_order = true;
        bool rush_always_one = true;
        bool saw_one = false;
        bool saw_two = false;
        bool first_order_seen[4] = {false, false, false, false};

        for (int attempt = 0; attempt < 200; attempt++)
        {
            game_state game = game_state();
            choose_mode(game, modes[m], 0);

            // Damage the orders, then start the mode again: it must make a fresh set.
            for (int i = 0; i < order_count_for_mode(modes[m]); i++)
            {
                game.orders[i].quantity = 99;
                game.orders[i].tier = -5;
            }
            choose_mode(game, modes[m], 0);
```

```
2974
2975            for (int i = 0; i < order_count_for_mode(modes[m]); i++)
2976            {
2977                if (!order_is_valid(game.orders[i], modes[m])) all_valid = false;
2978                if (game.orders[i].quantity == 99 || game.orders[i].tier == -5)
    old_orders_replaced = false;
2979                if (modes[m] == RUSH_MODE && game.orders[i].quantity != 1) rush_always_one =
    false;
2980                if (game.orders[i].quantity == 1) saw_one = true;
2981                if (game.orders[i].quantity == 2) saw_two = true;
2982            }
2983
2984            first_order_seen[game.orders[0].category] = true;
2985            if (!orders_use_each_category_once(game)) each_category_once = false;
2986            if (modes[m] == RECIPE_MODE && game.moves_left != game.orders[0].move_limit)
    moves_match_first_order = false;
2987        }
2988
2989        check(all_valid, names[m] + ": every random order has valid values");
2990        check(old_orders_replaced, names[m] + ": starting the mode makes a fresh set of
    orders");
2991        check(each_category_once, names[m] + ": the three orders use each food once");
2992        check(first_order_seen[DOUGH] && first_order_seen[TART] && first_order_seen[CAKE],
2993              names[m] + ": any food can be the first order (fair shuffle)");
2994
2995        if (modes[m] == RUSH_MODE)
2996            check(rush_always_one, "Rush Hour orders ask for one item");
2997        if (modes[m] == CLASSIC_MODE)
2998        {
2999            check(order_count_for_mode(CLASSIC_MODE) == 5, "Classic Bakery serves five
    customers");
3000            check(saw_one && saw_two, "Classic Bakery orders can ask for one or two items");
3001        }
3002        if (modes[m] == RECIPE_MODE)
3003            check(moves_match_first_order, "Recipe Challenge starts with the first order's
    move limit");
3004    }
3005
3006    // Beginner mode is the guided mode, so its orders never change.
3007    game_state game = game_state();
3008    choose_mode(game, BEGINNER_MODE, 0);
3009    check(game.orders[0].category == DOUGH && game.orders[1].category == TART &&
    game.orders[2].category == CAKE,
3010          "Beginner orders are always dough, tart, cake");
3011    check(game.orders[0].tier == 2 && game.orders[1].tier == 2 && game.orders[2].tier == 3,
3012          "Beginner order tiers are fixed");
3013}
3014
3015void test_item_tiers_and_random_squares()
3016{
3017    game_state game = game_state();
```

```
3018     choose_mode(game, BEGINNER_MODE, 0);
3019
3020     // Sixteen generator clicks must fill all sixteen squares (every empty square can be
     chosen).
3021     int created = 0;
3022     bool tiers_ok = true;
3023     for (int i = 0; i < 16; i++)
3024     {
3025         if (generate_item(game, (item_category)(DOUGH + i % 3))) created++;
3026     }
3027     for (int row = 0; row < 4; row++)
3028     {
3029         for (int col = 0; col < 4; col++)
3030         {
3031             if (game.board[row][col].category == NO_ITEM) created--;
3032             if (!is_in_range(game.board[row][col].tier, 0, MAX_ITEM_TIER)) tiers_ok = false;
3033         }
3034     }
3035     check(created == 16, "sixteen generator clicks fill the whole board");
3036     check(tiers_ok, "generated items stay within the tier limit");
3037     check(!generate_item(game, DOUGH), "a full board does not accept another item");
3038
3039     // When only one square is empty, that square must be the one chosen.
3040     game.board[3][2].category = NO_ITEM;
3041     game.board[3][2].tier = 0;
3042     check(generate_item(game, TART) && game.board[3][2].category == TART, "the only empty
     square is always chosen");
3043
3044     // With two empty squares, either of them can be chosen
3045     // (over 100 tries both are seen; the chance of missing one is far too small to matter).
3046     bool saw_first_square = false;
3047     bool saw_second_square = false;
3048     for (int attempt = 0; attempt < 100; attempt++)
3049     {
3050         empty_board(game);
3051         for (int row = 0; row < 4; row++)
3052             for (int col = 0; col < 4; col++)
3053                 place(game, row, col, DOUGH, 0);
3054         game.board[0][0].category = NO_ITEM;
3055         game.board[3][3].category = NO_ITEM;
3056         generate_item(game, TART);
3057         if (game.board[0][0].category == TART) saw_first_square = true;
3058         if (game.board[3][3].category == TART) saw_second_square = true;
3059     }
3060     check(saw_first_square && saw_second_square, "every empty square can be chosen,
     including the last one");
3061
3062     // Two foods already at the top tier cannot merge into a tier that has no picture.
3063     empty_board(game);
3064     place(game, 0, 0, DOUGH, MAX_ITEM_TIER);
3065     place(game, 0, 1, DOUGH, MAX_ITEM_TIER);
```

```
3066     select_or_move_cake(game, 0, 0, 0);
3067     select_or_move_cake(game, 0, 1, 0);
3068     check(game.board[0][0].tier == MAX_ITEM_TIER && game.board[0][1].tier == MAX_ITEM_TIER
&&
3069           game.board[0][0].category == DOUGH && game.board[0][1].category == DOUGH,
3070           "top-tier foods do not merge any further");
3071
3072     // Random play in every mode never creates a tier outside 0 to MAX_ITEM_TIER.
3073     game_mode modes[4] = {BEGINNER_MODE, CLASSIC_MODE, RUSH_MODE, RECIPE_MODE};
3074     bool all_in_range = true;
3075     for (int m = 0; m < 4; m++)
3076     {
3077         choose_mode(game, modes[m], 0);
3078         for (int step = 0; step < 1500; step++)
3079         {
3080             if (step % 3 == 0) generate_item(game, (item_category)(DOUGH + rnd(3)));
3081             select_or_move_cake(game, rnd(4), rnd(4), 0);
3082             update_game(game, 0);
3083             if (game.screen != PLAYING_SCREEN) choose_mode(game, modes[m], 0);
3084
3085             for (int row = 0; row < 4; row++)
3086                 for (int col = 0; col < 4; col++)
3087                     if (!is_in_range(game.board[row][col].tier, 0, MAX_ITEM_TIER))
all_in_range = false;
3088         }
3089     }
3090     check(all_in_range, "random play never makes a tier above MAX_ITEM_TIER");
3091 }
3092
3093 void test_settings_switches()
3094 {
3095     game_state game = game_state();
3096     start_new_game(game);
3097     check(game.music_on && game.sound_effects_on, "music and sound effects both start ON");
3098     check(!game.settings_open, "settings panel starts closed");
3099
3100     toggle_music(game);
3101     check(!game.music_on && game.sound_effects_on, "turning music off leaves sound effects
on");
3102     toggle_sound_effects(game);
3103     check(!game.music_on && !game.sound_effects_on, "both can be off at the same time");
3104     toggle_music(game);
3105     check(game.music_on && !game.sound_effects_on, "turning music on leaves sound effects
off");
3106     toggle_sound_effects(game);
3107     check(game.music_on && game.sound_effects_on, "both can be back on");
3108
3109     open_settings(game);
3110     check(game.settings_open, "open_settings shows the panel");
3111     close_settings(game);
3112     check(!game.settings_open, "close_settings hides the panel");
```

```
3113
3114        // Settings stay until the player changes them.
3115        toggle_music(game);   // music off, effects on
3116        show_mode_menu(game);
3117        check(!game.music_on && game.sound_effects_on, "settings survive going back to the
    menu");
3118        choose_mode(game, RUSH_MODE, 0);
3119        game.screen = GAME_OVER_SCREEN;
3120        show_mode_menu(game);
3121        choose_mode(game, RECIPE_MODE, 0);
3122        check(!game.music_on && game.sound_effects_on, "settings survive a whole round");
3123
3124        open_settings(game);
3125        show_mode_menu(game);
3126        check(!game.settings_open, "returning to the menu closes the panel");
3127    }
3128
3129    // Click on the settings button, switches and Close button the way a player would.
3130    void click(game_state &game, int x, int y)
3131    {
3132        handle_mouse_click(game, x, y, 0);
3133    }
3134
3135    void test_settings_panel_clicks()
3136    {
3137        game_state game = game_state();
3138        start_new_game(game);
3139        show_mode_menu(game);
3140
3141        click(game, SETTINGS_BUTTON_X + 5, SETTINGS_BUTTON_Y + 5);
3142        check(game.settings_open && game.screen == MODE_MENU, "Settings button opens the
    panel");
3143
3144        click(game, SETTINGS_TOGGLE_X + 10, SETTINGS_MUSIC_Y + 10);
3145        check(!game.music_on && game.sound_effects_on, "music switch only changes music");
3146        click(game, SETTINGS_TOGGLE_X + 10, SETTINGS_EFFECTS_Y + 10);
3147        check(!game.music_on && !game.sound_effects_on, "effects switch only changes sound
    effects");
3148        click(game, SETTINGS_TOGGLE_X + 10, SETTINGS_MUSIC_Y + 10);
3149        check(game.music_on && !game.sound_effects_on, "music can be switched back on by
    itself");
3150
3151        // While the panel is open the mode cards behind it cannot be clicked.
3152        click(game, MODE_LEFT_X + 10, MODE_TOP_Y + 10);
3153        click(game, SETTINGS_BOX_X - 5, SETTINGS_BOX_Y - 5);
3154        check(game.screen == MODE_MENU && game.settings_open, "clicks outside the panel do
    nothing");
3155
3156        click(game, SETTINGS_CLOSE_X + 10, SETTINGS_CLOSE_Y + 10);
3157        check(!game.settings_open && game.screen == MODE_MENU, "Close button hides the panel");
3158        check(game.music_on && !game.sound_effects_on, "closing the panel keeps the settings");
```

```
3159
3160        click(game, MODE_LEFT_X + 10, MODE_TOP_Y + 10);
3161        check(game.screen == PLAYING_SCREEN && game.mode == BEGINNER_MODE, "mode cards work
    again after closing");
3162        check(game.music_on && !game.sound_effects_on, "settings are unchanged during play");
3163    }
3164
3165    int main()
3166    {
3167        load_game_sounds(); // the game logic plays sound effects
3168
3169        test_utilities();
3170        test_layout();
3171        test_screen_flow();
3172        test_generators();
3173        test_move_and_merge();
3174        test_order_complete();
3175        test_recipe_rejects_higher_tier();
3176        test_recipe_food_collection_delay();
3177        test_beginner_has_no_high_mark();
3178        test_customer_leaving();
3179        test_prebuilt_items_count_for_all_modes();
3180        test_final_order_wins();
3181        test_rush_hour_timer();
3182        test_recipe_move_limit();
3183        test_served_food_disappears();
3184        test_hearts_only_in_rush_hour();
3185        test_back_to_menu();
3186        test_random_orders();
3187        test_item_tiers_and_random_squares();
3188        test_settings_switches();
3189        test_settings_panel_clicks();
3190
3191        return 0;
3192    }
```